```
    // Usage - Type inferred automatically
    String s = Utils.identity("Hello");  // T = String
    Integer i = Utils.identity(123);     // T = Integer
}

// Example 2: Generic Method with Multiple Parameters
public class Utils {
    public static <T> boolean areEqual(T a, T b) {
        return a.equals(b);
    }
     // Usage
    boolean same = Utils.areEqual("A", "A");  // true
    boolean diff = Utils.areEqual(1, 2);     // false
}
// Example 3: Generic Method in Non-Generic Class
public class ArrayUtils {
    // Entire class is NOT generic, but method is
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.println(element);
        }
    }
    // Usage
    String[] strings = {"A", "B", "C"};
    ArrayUtils.printArray(strings);  // T = String

    Integer[] numbers = {1, 2, 3};
    ArrayUtils.printArray(numbers);  // T = Integer
}
```

```
// Example 4: Generic Method with Return Type Transformation
public class Converter {
    public static <T, R> R convert(T input, Function<T, R> converter) {
        return converter.apply(input);
    }

    // Usage
    String numStr = "123";
    Integer num = Converter.convert(numStr, Integer::parseInt);
}

// Example 5: Generic Method with Variable Arguments
public class Collections {
    public static <T> List<T> asList(T... elements) {
        List<T> list = new ArrayList<>();
        for (T element : elements) {
            list.add(element);
        }
        return list;
    }

    // Usage
    List<String> strings = Collections.asList("A", "B", "C");
    List<Integer> numbers = Collections.asList(1, 2, 3);
}

// Example 6: Generic Method with Bounded Type
public class NumberUtils {
    public static <T extends Number> double sum(List<T> numbers) {
```

```
    double total = 0;
    for (T num : numbers) {
        total += num.doubleValue();  // Safe - T extends Number
    }
    return total;
  }

  // Usage
  List<Integer> ints = Arrays.asList(1, 2, 3);
  double total = NumberUtils.sum(ints);  // 6.0

  // List<String> strings = Arrays.asList("A", "B");
  // NumberUtils.sum(strings);  // ❌ Compiler ERROR!
}
```

1.7 Generic Method Rules

Rule	Description
Type Declaration	<T> must come BEFORE return type
Static Methods	Can be generic even in non-generic class
Type Inference	Compiler infers type from arguments
Explicit Type	Can specify explicitly: Utils.<String>identity("A")
Multiple Types	Can have multiple: <K, V> method(K k, V v)

PART 4: BOUNDED TYPE PARAMETERS

1. What are Bounded Types?

1.1 Definition

Bounded Type Parameters restrict the types that can be used as type arguments. This allows you to call methods on the type parameter safely.

1.2 Syntax

```
// Upper bounded - T must be subtype of Number

class Box<T extends Number> { }

// Multiple bounds - T must extend A AND implement B, C

class Box<T extends A & B & C> { }
```

1.3 Why Bounded Types?

```
┌──────────────────────────────────────────────────────┐
│              WHY BOUNDED TYPES?                      │
├──────────────────────────────────────────────────────┤
│                                                      │
│  Problem:                                            │
│  public class Box<T> {                               │
│      private T value;                                │
│      public double getDouble() {                     │
│          return value.doubleValue();  // ❌ ERROR!   │
│      }                                               │
│  }                                                   │
│                                                      │
│  Why Error? T could be ANY type (String, Object, etc.)  │
│  Not all types have doubleValue() method             │
│                                                      │
│  Solution:                                           │
│  public class Box<T extends Number> {                │
│      private T value;                                │
│      public double getDouble() {                     │
│          return value.doubleValue();  // ✅ OK!      │
│      }                                               │
│  }                                                   │
│                                                      │
│  Now T is guaranteed to be Number or subclass        │
│  All Number subclasses have doubleValue()            │
│                                                      │
└──────────────────────────────────────────────────────┘
```

1.4 Types of Bounds

Upper Bounded (extends)

```
// T must be Number or subclass (Integer, Double, etc.)

public class NumberBox<T extends Number> {

   private T value;


   public double getDoubleValue() {
```

```
        return value.doubleValue();  // Safe!
    }
}

// Valid
NumberBox<Integer> intBox = new NumberBox<>();
NumberBox<Double> doubleBox = new NumberBox<>();
NumberBox<Float> floatBox = new NumberBox<>();

// Invalid
// NumberBox<String> stringBox = new NumberBox<>(); // ❌ ERROR!
```

Multiple Bounds

```
// T must extend Comparable AND implement Serializable
public class SortedBox<T extends Comparable<T> & Serializable> {
    private T value;

    public int compareTo(T other) {
        return value.compareTo(other);  // Safe - T extends Comparable
    }
}

// Rules:
// - First bound must be class (if any)
// - Rest must be interfaces
// - Class bound must come first
```

Lower Bounded (in Wildcards)

java

```
// See Wildcards section for <? super T>
```

PART 5: WILDCARDS

1. What are Wildcards?

1.1 Definition

Wildcards (?) represent an **unknown type**. They are used when you want to write code that works with any type, but with specific constraints.

1.2 Types of Wildcards

Wildcard	Syntax	Meaning	Use Case
Unbounded	?	Any type	Read-only operations
Upper Bounded	? extends T	T or subtype	Read-only (PECS: Producer)
Lower Bounded	? super T	T or supertype	Write-only (PECS: Consumer)

1.3 The PECS Principle

```
┌──────────────────────────────────────────────────────┐
│                    PECS PRINCIPLE                    │
├──────────────────────────────────────────────────────┤
│                                                      │
│  PECS = Producer Extends, Consumer Super             │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │  If parameter PRODUCES values (you read from it):  │
│  │  → Use <? extends T>                         │    │
│  │  Example: List<? extends Number> - read Numbers    │
│  └──────────────────────────────────────────────┘    │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │  If parameter CONSUMES values (you write to it):   │
│  │  → Use <? super T>                           │    │
│  │  Example: List<? super Integer> - write Integers   │
│  └──────────────────────────────────────────────┘    │
│                                                      │
│  Mnemonic:                                           │
│  - Producer → Extends (you get values OUT)           │
│  - Consumer → Super (you put values IN)              │
│                                                      │
└──────────────────────────────────────────────────────┘
```

1.4 Internal Working of Wildcards

```
┌──────────────────────────────────────────────────────────┐
│                WILDCARD INTERNAL WORKING                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Unbounded Wildcard (?):                                 │
│  ┌──────────────────────────────────────────────────┐    │
│  │  List<?> list = new ArrayList<String>();         │    │
│  │                                                  │    │
│  │  After Erasure: List<Object> (effectively)       │    │
│  │                                                  │    │
│  │  Can read: Object (most general type)            │    │
│  │  Can write: null only                            │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  Upper Bounded (? extends T):                            │
│  ┌──────────────────────────────────────────────────┐    │
│  │  List<? extends Number> list = new ArrayList<>();  │    │
│  │                                                  │    │
│  │  Actual type could be:                           │    │
│  │  - List<Integer>                                 │    │
│  │  - List<Double>                                  │    │
│  │  - List<Float>                                   │    │
│  │                                                  │    │
│  │  Can read: Number (common supertype)             │    │
│  │  Can write: null only (don't know exact type)    │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  Lower Bounded (? super T):                              │
│  ┌──────────────────────────────────────────────────┐    │
│  │  List<? super Integer> list = new ArrayList<>();   │    │
│  │                                                  │    │
│  │  Actual type could be:                           │    │
│  │  - List<Integer>                                 │    │
│  │  - List<Number>                                  │    │
│  │  - List<Object>                                  │    │
│  │                                                  │    │
│  │  Can read: Object (most general type)            │    │
│  │  Can write: Integer or subtype                   │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2. Unbounded Wildcard (?)

2.1 Definition

Represents a list of **any type**.

2.2 When to Use

- When you only need to read from the collection
- When you don't care about the actual type
- For methods that work with any collection type

2.3 How to Use

```
// Method that prints any list
public static void printList(List<?> list) {
  for (Object element : list) {
    System.out.println(element);
  }
}
```

```
// Usage
List<String> strings = Arrays.asList("A", "B", "C");
List<Integer> numbers = Arrays.asList(1, 2, 3);
printList(strings);  // OK
printList(numbers);  // OK
// What you CAN do:
Object obj = list.get(0);  // Read as Object
int size = list.size();    // Get size
// What you CANNOT do:
// list.add("Hello");  // ❌ ERROR! Don't know the type
// list.add(null);     // ✅ OK (null works with any type)
```

2.4 Internal Limitations

```
┌──────────────────────────────────────────────┐
│            UNBOUNDED WILDCARD LIMITATIONS     │
├──────────────────────────────────────────────┤
│                                               │
│ List<?> list = new ArrayList<String>();       │
│                                               │
│ Compiler sees: List of UNKNOWN type           │
│                                               │
│ Question: Can we add a String?                │
│ Answer: NO! Because list could be List<Integer>│
│                                               │
│ Question: Can we add an Integer?              │
│ Answer: NO! Because list could be List<String>│
│                                               │
│ Question: Can we add null?                    │
│ Answer: YES! null works with any type         │
│                                               │
│ Question: What type can we read?              │
│ Answer: Object (the only common supertype)    │
│                                               │
└──────────────────────────────────────────────┘
```

3. Upper Bounded Wildcard (? extends T)

3.1 Definition

Represents a list of **T or any subtype of T**.

3.2 When to Use

- When you need to **READ** values from a collection
- When the method should accept List of T or any subclass
- **Producer** scenario (PECS)

3.3 How to Use

```
// Method that sums numbers from any Number list
public static double sumList(List<? extends Number> list) {
   double sum = 0;
   for (Number n : list) {  // Can read as Number
      sum += n.doubleValue();
   }
   return sum;
}

// Usage
List<Integer> integers = Arrays.asList(1, 2, 3);
List<Double> doubles = Arrays.asList(1.5, 2.5, 3.5);
List<Float> floats = Arrays.asList(1.0f, 2.0f);
sumList(integers);  // OK - Integer extends Number
sumList(doubles);   // OK - Double extends Number
sumList(floats);    // OK - Float extends Number
// What you CAN do:
Number n = list.get(0);    // Read as Number (or supertype)
Double d = list.get(0);    // If you know it's Double
// What you CANNOT do:
// list.add(5);       // ❌ ERROR! Don't know exact type
// list.add(new Integer(5)); // ❌ ERROR!
```

3.4 Why Can't We Write?

```
┌──────────────────────────────────────────────────┐
│         WHY ? extends T PREVENTS WRITING          │
├──────────────────────────────────────────────────┤
│                                                   │
│  Scenario:                                        │
│  List<? extends Number> list = new ArrayList<Integer>();  │
│                                                   │
│  Question: Can we add a Double?                   │
│  Answer: NO! Because actual list is List<Integer>  │
│                                                   │
│  Question: Can we add an Integer?                 │
│  Answer: NO! Because compiler doesn't know it's Integer  │
│          It could be List<Double> at runtime      │
│                                                   │
│  The compiler plays it safe:                      │
│  - It knows list contains Number or subclass      │
│  - It doesn't know WHICH subclass                 │
│  - So it prevents ALL writes (except null)        │
│                                                   │
│  This is called "Capture Conversion"              │
│                                                   │
└──────────────────────────────────────────────────┘
```

3.5 Upper Bounded Examples

```
// Example 1: Processing different animal types
class Animal { }
class Dog extends Animal { }
class Cat extends Animal { }

public class Zoo {
  // Can process List<Dog>, List<Cat>, List<Animal>
  public static void printAnimals(List<? extends Animal> animals) {
    for (Animal a : animals) {  // Can read as Animal
      System.out.println(a);
    }
  }
}

// Example 2: Copy method
public class Collections {
  // Copy from source to destination
  public static <T> void copy(List<? super T> dest,
```

```
                    List<? extends T> src) {
    for (T item : src) {  // Read from src (producer)
      dest.add(item);    // Write to dest (consumer)
    }
  }
}

// Example 3: Finding max
public class Algorithms {
  public static <T extends Comparable<? super T>>
      T max(List<? extends T> list) {
    T max = list.get(0);
    for (T item : list) {
      if (item.compareTo(max) > 0) {
        max = item;
      }
    }
    return max;
  }
}
```

4. Lower Bounded Wildcard (? super T)

4.1 Definition

Represents a list of **T or any supertype of T**.

4.2 When to Use

- When you need to **WRITE** values to a collection
- When the method should accept List of T or any superclass
- **Consumer** scenario (PECS)

4.3 How to Use

```
// Method that adds integers to any compatible list
public static void addNumbers(List<? super Integer> list) {
```

```
    list.add(1);      // ✅ OK - Integer is T
    list.add(2);      // ✅ OK
    // list.add(1.5);  // ❌ ERROR - Double not compatible
}
// Usage
List<Integer> integers = new ArrayList<>();
List<Number> numbers = new ArrayList<>();
List<Object> objects = new ArrayList<>();
addNumbers(integers);  // OK - Integer is Integer
addNumbers(numbers);   // OK - Integer is subtype of Number
addNumbers(objects);   // OK - Integer is subtype of Object
// What you CAN do:
list.add(5);        // Write Integer or subtype
list.add(null);      // Write null

// What you CANNOT do (safely):
// Integer i = list.get(0);  // ❌ ERROR! Could be List<Object>
// Object o = list.get(0);   // ✅ OK - Object is supertype of all
```

4.4 Why Reading Returns Object?

```
┌──────────────────────────────────────────────────┐
│         WHY ? super T RETURNS OBJECT WHEN READING  │
├──────────────────────────────────────────────────┤
│                                                  │
│ Scenario:                                        │
│ List<? super Integer> list = new ArrayList<Object>();  │
│                                                  │
│ Question: What type can we read?                 │
│ Answer: Only Object!                             │
│                                                  │
│ Why?                                             │
│ - List could be List<Integer> → can read Integer │
│ - List could be List<Number> → can read Number   │
│ - List could be List<Object> → can read Object   │
│                                                  │
│ The only common type across all possibilities: Object  │
│                                                  │
│ So compiler allows:                              │
│ Object o = list.get(0);  // ✅ OK                 │
│ Number n = list.get(0);  // ❌ ERROR!             │
│ Integer i = list.get(0); // ❌ ERROR!             │
│                                                  │
└──────────────────────────────────────────────────┘
```

4.5 Lower Bounded Examples

```
// Example 1: Adding to collection
public class NumberAdder {
   public static void addIntegers(List<? super Integer> list) {
      for (int i = 1; i <= 10; i++) {
         list.add(i);  // Can add Integer
      }
   }
}

// Example 2: Comparator
public class Sorter {
   // Comparator that can compare T or any supertype
   public static <T> void sort(List<T> list,
                               Comparator<? super T> comp) {
      Collections.sort(list, comp);
   }
}

// Example 3: Consumer pattern
public class Processor {
   public static void process(List<? super String> list,
                              String value) {
      list.add(value);  // Can write String
   }
}
```

---

5. Wildcard Comparison Table

Wildcard	Syntax	Read As	Write	Use Case
Unbounded	List<?>	Object	null only	Any type, read-only
Upper Bounded	List<? extends T>	T (or supertype)	null only	Producer (read T or subtype)
Lower Bounded	List<? super T>	Object	T (or subtype)	Consumer (write T or supertype)
No Wildcard	List<T>	T	T	Both read and write

6. PECS Examples - Complete

```
// Producer Extends, Consumer Super


public class PECSExamples {


  // PRODUCER - You READ from it
  public static double sum(List<? extends Number> numbers) {
    double sum = 0;
    for (Number n : numbers) {  // Read as Number
      sum += n.doubleValue();
    }
    return sum;
    // Cannot add to numbers (producer)
  }
    // CONSUMER - You WRITE to it
  public static void addNumbers(List<? super Integer> list) {
    list.add(1);   // Write Integer
    list.add(2);
    // Can only read as Object (not useful)
  }


  // BOTH - No wildcard needed
```

```
    public static void process(List<Number> list) {
        list.add(5);        // Write
        Number n = list.get(0); // Read
    }
    // Real-world example: Collections.copy
    public static <T> void copy(List<? super T> dest,
                                List<? extends T> src) {
        // src is producer (read from it)
        // dest is consumer (write to it)
        for (T item : src) {
            dest.add(item);
        }
    }
}
```

PART 6: TYPE ERASURE - INTERNAL WORKING

1. What is Type Erasure?

1.1 Definition

Type Erasure is the process where the Java compiler removes all generic type information during compilation. At runtime, generic types are replaced with their bounds (or Object if unbounded).

1.2 Why Type Erasure Exists?

```
┌──────────────────────────────────────────────────────────┐
│                    WHY TYPE ERASURE?                     │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Historical Reason:                                      │
│  - Generics added in Java 5 (2004)                       │
│  - Needed to be backward compatible with Java 1.4 code   │
│  - Old code: List list = new ArrayList();                │
│  - New code: List<String> list = new ArrayList<>();      │
│                                                          │
│  Solution:                                               │
│  - Both compile to same bytecode: List                   │
│  - Old code continues to work                            │
│  - No runtime overhead for generics                      │
│                                                          │
│  Trade-off:                                              │
│  - ✅ Backward compatible                                │
│  - ✅ No runtime performance cost                        │
│  - ❌ Type info not available at runtime                 │
│  - ❌ Cannot create generic arrays                       │
│  - ❌ Cannot use instanceof with generics                │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.3 Type Erasure Process

```
┌──────────────────────────────────────────────────────────┐
│                TYPE ERASURE PROCESS                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Step 1: Replace type parameters with bounds             │
│  ┌──────────────────────────────────────────────────┐    │
│  │  Before: public class Box<T> {                   │    │
│  │          private T value;                        │    │
│  │          public T get() { return value; }        │    │
│  │  }                                               │    │
│  │                                                  │    │
│  │  After:  public class Box {                      │    │
│  │          private Object value;  // T → Object    │    │
│  │          public Object get() { return value; }   │    │
│  │  }                                               │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  Step 2: Insert casts where needed                       │
│  ┌──────────────────────────────────────────────────┐    │
│  │  Before: Box<String> box = new Box<>();          │    │
│  │          String s = box.get();                   │    │
│  │                                                  │    │
│  │  After:  Box box = new Box();                    │    │
│  │          String s = (String) box.get();  // Cast!    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  Step 3: Preserve overloaded methods                     │
│  ┌──────────────────────────────────────────────────┐    │
│  │  Before: void method(List<String> list) { }      │    │
│  │          void method(List<Integer> list) { }     │    │
│  │                                                  │    │
│  │  After:  ERROR! Both become method(List list)    │    │
│  │          Cannot have duplicate methods           │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.4 Type Erasure Rules

Scenario	Before Erasure	After Erasure
Unbounded type	<T>	Object
Bounded type	<T extends Number>	Number
Multiple bounds	<T extends A & B>	First bound A
Generic method	<T> T method(T t)	Object method(Object t)
Generic field	private T value	private Object value

1.5 Type Erasure Examples

// Example 1: Simple Generic Class

```
// BEFORE
public class Box<T> {
   private T value;
   public T get() { return value; }
   public void set(T value) { this.value = value; }
}
```

```
// AFTER (what JVM sees)
public class Box {
   private Object value;
   public Object get() { return value; }
   public void set(Object value) { this.value = value; }
}
// Example 2: Bounded Type
// BEFORE
public class NumberBox<T extends Number> {
   private T value;
   public double getDouble() {
      return value.doubleValue();
   }
}
// AFTER
public class NumberBox {
   private Number value;  // T → Number
   public double getDouble() {
      return value.doubleValue();
   }
}
// Example 3: Generic Method
// BEFORE
public static <T> T identity(T value) {
   return value;
}
// AFTER
public static Object identity(Object value) {
   return value;
}
```

2. Limitations Due to Type Erasure

2.1 Cannot Create Generic Arrays

```java
// ❌ ERROR - Cannot create array of generic type
List<String>[] array = new List<String>[10];

// Why? After erasure:
// List[] array = new List[10];
// Loses type safety - could add List<Integer>

// ✅ Workaround - Use ArrayList
List<List<String>> listOfLists = new ArrayList<>();
```

2.2 Cannot Use instanceof with Generics

```java
// ❌ ERROR - Cannot check generic type at runtime
if (obj instanceof List<String>) { }

// Why? After erasure, all List<String> become List
// Runtime doesn't know it was String

// ✅ Workaround - Check raw type
if (obj instanceof List) { }
```

2.3 Cannot Create Generic Exception Classes

```java
// ❌ ERROR
public class MyException<T> extends Exception { }

// Why? Exception handling relies on runtime type info
// Which is erased for generics
```

2.4 Cannot Overload Methods with Same Erased Signature

```java
// ❌ ERROR - Both methods erase to same signature
public void method(List<String> list) { }
public void method(List<Integer> list) { }

// After erasure, both become:
// public void method(List list) { }
// Duplicate method definition!
```

2.5 Static Fields Cannot Use Type Parameter

```java
// ❌ ERROR
public class Box<T> {
    private static T value;  // Static context, no T available
}

// Why? Static fields belong to class, not instance
// Type parameter is per-instance

// ✅ Workaround - Use Object
public class Box<T> {
    private static Object value;
}
```

PART 7: COMMON PITFALLS & BEST PRACTICES

1. Common Pitfalls

1.1 Raw Types

```java
// ❌ AVOID - Raw type (no type parameter)
List list = new ArrayList();
list.add("Hello");
list.add(123);  // Compiles but unsafe!

// ✅ USE - Parameterized type
List<String> list = new ArrayList<>();
list.add("Hello");
// list.add(123);  // Compiler error!
```

1.2 Generic Array Creation

```java
// ❌ ERROR
List<String>[] array = new List<String>[10];

// ✅ Workaround
List<List<String>> listOfLists = new ArrayList<>();
```

1.3 instanceof with Generics

```java
// ❌ ERROR
if (obj instanceof List<String>) { }

// ✅ Workaround
if (obj instanceof List) {
    List<?> list = (List<?>) obj;
}
```

3. Workarounds for Type Erasure Limitations

3.1 Class Token Pattern

```java
// Pass Class<T> to preserve type info at runtime
public class Box<T> {
    private Class<T> type;
    private T value;

    public Box(Class<T> type) {
        this.type = type;
    }

    public T create() {
        try {
            return type.newInstance();  // Can create instance
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }

    public boolean isInstance(Object obj) {
        return type.isInstance(obj);  // Can check type
    }
}

// Usage
Box<String> stringBox = new Box<>(String.class);
String s = stringBox.create();
boolean check = stringBox.isInstance("Hello");  // true
```

3.2 Generic Array Creation Workaround

```java
// Using reflection
@SuppressWarnings("unchecked")
public static <T> T[] createArray(Class<T> type, int size) {
    return (T[]) Array.newInstance(type, size);
}

// Usage
String[] array = createArray(String.class, 10);
```

PART 7: COMMON PITFALLS & BEST PRACTICES

1. Common Pitfalls

1.1 Raw Types

```java
// ❌ AVOID - Raw type (no type parameter)
List list = new ArrayList();
list.add("Hello");
list.add(123);  // Compiles but unsafe!

// ✅ USE - Parameterized type
List<String> list = new ArrayList<>();
list.add("Hello");
// list.add(123);  // Compiler error!
```

1.2 Generic Array Creation

```java
// ❌ ERROR
List<String>[] array = new List<String>[10];

// ✅ Workaround
List<List<String>> listOfLists = new ArrayList<>();
```

1.3 instanceof with Generics

```java
// ❌ ERROR
if (obj instanceof List<String>) { }

// ✅ Workaround
if (obj instanceof List) {
    List<?> list = (List<?>) obj;
}
```

1.4 Mixing Wildcards Incorrectly

```java
// ❌ Confusing - Don't mix extends and super unnecessarily
public void method(List<? extends Number> list) {
    // list.add(5);  // ERROR - can't write
}

// ✅ Clear intent
// If you need to read: ? extends T
// If you need to write: ? super T
// If you need both: No wildcard, use T
```

1.5 Returning Wildcard Types

```java
// ❌ AVOID - Wildcard in return type
public List<? extends Number> getNumbers() { }

// ✅ BETTER - Use concrete type or type parameter
public List<Number> getNumbers() { }
public <T extends Number> List<T> getNumbers() { }
```

```
Need to use Generics?
│
├─ Creating a class/interface?
│   └─ Use Generic Class: class Box<T>
│
├─ Creating a method?
│   ├─ Method needs own type param?
│   │   └─ Use Generic Method: <T> void method(T t)
│   └─ Method in generic class?
│       └─ Use class type param: void method(T t)
│
├─ Need to constrain type?
│   └─ Use Bounded Type: <T extends Number>
│
├─ Method parameter - reading values?
│   └─ Use Upper Wildcard: List<? extends T>
│
├─ Method parameter - writing values?
│   └─ Use Lower Wildcard: List<? super T>
│
└─ Need both read and write?
    └─ Use Concrete Type: List<T>
```

5. Interview Questions on Generics

Question	Answer
What is type erasure?	Compiler removes generic type info at compile time
Why can't we create generic arrays?	Type info erased at runtime, breaks type safety
What is PECS?	Producer Extends, Consumer Super
Difference between `List<T>` and `List<?>`?	`List<T>` is specific type, `List<?>` is any type
Can we overload methods with generic types?	Not if they erase to same signature
What is bounded type parameter?	Type parameter with constraint (`extends`)
Why use wildcards?	For API flexibility with type safety
Can static methods use class type parameter?	No, static context doesn't have type param

Practice	Description
Use Generics	Always parameterize collections
Avoid Raw Types	Never use List without <T>
Follow PECS	Producer extends, Consumer super
Use Diamond Operator	new ArrayList<>() not new ArrayList<String>()
Prefer Type Parameters	Over wildcards when possible
Document Bounds	Clearly document type constraints
Avoid Generic Arrays	Use List<T> instead
Use Class Tokens	When runtime type info needed

---

3. Complete Generics Reference Table

Concept	Syntax	Use Case	Read	Write
Concrete Type	List<T>	Both read/write	T	T
Unbounded Wildcard	List<?>	Any type	Object	null
Upper Bound	List<? extends T>	Producer (read)	T	null
Lower Bound	List<? super T>	Consumer (write)	Object	T
Bounded Type Param	<T extends Number>	Constrain type	T	T
Multiple Bounds	<T extends A & B>	Multiple constraints	T	T

```
// At call site:

// String s = identity("Hello");

// Compiled to: String s = (String) identity((Object)"Hello");
```

File Handling (I/O Streams) - Complete Deep Dive

File Handling (Input/Output) is fundamental for persisting data, reading configurations, processing logs, and communicating with external systems. This module covers the classic java.io package in depth.

Table of Contents

1. **Introduction to I/O** - Streams, Sources, Sinks
2. **The File Class** - Path abstraction
3. **Stream Hierarchy** - Byte vs. Character
4. **Byte Streams** - FileInputStream, FileOutputStream
5. **Character Streams** - FileReader, FileWriter
6. **Buffered Streams** - BufferedReader, BufferedWriter
7. **Scanner Class** - Parsing input
8. **Serialization** - Object persistence
9. **Internal Working Deep Dive** - OS interaction, Buffering, Serialization Graph
10. **Best Practices & Pitfalls**

PART 1: INTRODUCTION TO I/O

1. What is Java I/O?

Definition: Java I/O (Input/Output) is a mechanism to process data. **Input** means reading data from a source (file, network, console). **Output** means writing data to a destination (file, network, console).

2. What is a Stream?

Definition: A **Stream** is an abstract representation of a sequence of data elements. It allows data to flow from a source to a destination.

```
┌──────────────────────────────────────────────┐
│                  STREAM FLOW                 │
├──────────────────────────────────────────────┤
│                                              │
│  Source (Input)  →  [ Stream ]  →  Destination (Output)  │
│                                              │
│  Examples:                                   │
│  - File → [ InputStream ] → Memory (Reading) │
│  - Memory → [ OutputStream ] → File (Writing) │
│  - Keyboard → [ InputStream ] → Program      │
│  - Program → [ OutputStream ] → Screen       │
│                                              │
└──────────────────────────────────────────────┘
```

3. Why Two Types of Streams?

Feature	Byte Streams	Character Streams
Unit	8-bit Bytes	16-bit Characters (Unicode)
Classes	InputStream, OutputStream	Reader, Writer
Use Case	Binary data (images, audio, video)	Text data (files, strings)
Encoding	No encoding handling	Handles Charset encoding (UTF-8, etc.)
Performance	Faster for binary	Slower (due to encoding conversion)

PART 2: THE FILE CLASS

1. What is the File Class?

Definition: java.io.File is an abstraction of a **file or directory path**. It does **not** represent the content of the file, only its metadata and location on the disk.

2. Why We Use It?

- To check if a file exists.
- To create/delete files or directories.
- To get file properties (size, permissions, path).
- To list files in a directory.

3. When & Where to Use?

- ✅ **Before I/O:** Check if file exists before reading.
- ✅ **Setup:** Create directories before writing logs.
- ✅ **Cleanup:** Delete temporary files after processing.

4. How to Use? (Methods)

```
import java.io.File;

import java.io.IOException;

public class FileExample {

  public static void main(String[] args) {

    // Create File object (does not create actual file on disk)

    File file = new File("test.txt");

    File dir = new File("myFolder");
```

```
    // Check existence
    boolean exists = file.exists();  // false
    // Create actual file
    try {
      boolean created = file.createNewFile();  // true if created
    } catch (IOException e) { e.printStackTrace(); }
    // Create directory
    dir.mkdir();   // Single level
    dir.mkdirs();  // Multiple levels (parents)
    // Get Info
    String name = file.getName();     // "test.txt"
    String path = file.getAbsolutePath();
    long size = file.length();        // Size in bytes
    boolean isFile = file.isFile();    // true
    boolean isDir = file.isDirectory(); // false
    // List files in directory
    File[] files = dir.listFiles();

    // Delete
    file.delete();  // Returns true if successful
  }
}
```

5. Internal Working

- **Abstraction:** File objects are platform-dependent.
  - Windows: C:\Users\file.txt
  - Linux: /home/user/file.txt
- **System Calls:** Methods like exists(), createNewFile() invoke **native OS system calls** (e.g., stat, open, unlink in Linux).
- **No Content:** File object holds only the **path string**. It does not open a handle to the file content until an Stream/Reader is created.

PART 3: BYTE STREAMS

1. FileInputStream

1.1 What is it?

- **Definition:** Reads raw **bytes** from a file.
- **Hierarchy:** Extends InputStream.
- **Use Case:** Images, audio, video, serialized objects, any non-text file.

1.2 Why We Use?

- To read binary data exactly as stored.
- No character encoding conversion (fastest for binary).

1.3 Methods

Method	Description	Returns
read()	Reads single byte	int (0-255) or -1 (EOF)
read(byte[] b)	Reads into array	int (bytes read)
read(byte[] b, int off, int len)	Reads portion	int
skip(long n)	Skips bytes	long
available()	Estimated bytes remaining	int
close()	Closes stream	void

1.4 How to Use?

```
import java.io.*;

public class ByteRead {

  public static void main(String[] args) {

    // Try-with-resources (Auto-close)

    try (FileInputStream fis = new FileInputStream("image.jpg")) {

      int content;

      // Read byte by byte (Slow for large files)

      while ((content = fis.read()) != -1) {

        System.out.print((char) content);

      }
```

```
            // Better: Read in chunks
        byte[] buffer = new byte[1024];
        int bytesRead;
        while ((bytesRead = fis.read(buffer)) != -1) {
          // Process buffer
        }
      } catch (IOException e) { e.printStackTrace(); }
    }
}
```

1.5 Internal Working

- **File Descriptor:** Opens a handle to the OS file descriptor.
- **Native Method:** read() calls native code (e.g., C read() function).
- **Blocking:** read() blocks until data is available or EOF.
- **Position:** Maintains an internal file pointer that advances after each read.

---

2. FileOutputStream

2.1 What is it?

- **Definition:** Writes raw **bytes** to a file.
- **Hierarchy:** Extends OutputStream.

2.2 Constructors

```
new FileOutputStream("file.txt");      // Overwrites
new FileOutputStream("file.txt", true);  // Appends (true)
```

2.3 Methods

Method	Description
write(int b)	Writes single byte
write(byte[] b)	Writes array
write(byte[] b, int off, int len)	Writes portion
flush()	Forces write to disk (buffered streams)
close()	Closes stream

2.4 How to Use?

```
try (FileOutputStream fos = new FileOutputStream("output.bin")) {
   String data = "Hello";
   fos.write(data.getBytes());  // Convert to bytes
   fos.write(65);            // Writes 'A'
}
 catch (IOException e) {
e.printStackTrace();
}
```

2.5 Internal Working

- **Buffering:** Basic FileOutputStream has minimal buffering. Data is written to OS kernel buffer.
- **Flush:** flush() forces kernel buffer to write to physical disk.
- **Append Mode:** If append=true, seeks to end of file before writing.

PART 4: CHARACTER STREAMS

1. FileReader

1.1 What is it?

- **Definition:** Reads **characters** from a file.
- **Hierarchy:** Extends InputStreamReader → Reader.
- **Use Case:** Text files (.txt, .csv, .java, .xml).

1.2 Why We Use?

- Handles **Character Encoding** (UTF-8, ASCII, etc.).
- Converts bytes to characters automatically.
- Easier to read text than byte streams.

1.3 Internal Working

- **Wrapper:** FileReader is essentially new FileInputStream(file, charset).
- **Decoding:** Uses CharsetDecoder to convert byte sequences into Unicode characters.
- **Default Charset:** Uses platform default encoding (risky! Can differ between Windows/Linux).

1.4 How to Use?

java

```
try (FileReader fr = new FileReader("text.txt")) {
   int content;
   while ((content = fr.read()) != -1) {
      System.out.print((char) content);
   }
} catch (IOException e)
 {
e.printStackTrace();
 }
```

2. FileWriter

2.1 What is it?

- **Definition:** Writes **characters** to a file.
- **Hierarchy:** Extends OutputStreamWriter → Writer.

2.2 Internal Working

- **Encoding:** Converts characters to bytes using specific Charset before writing to disk.
- **Default Charset:** Uses platform default (same risk as FileReader).

2.3 How to Use?

```
try (FileWriter fw = new FileWriter("text.txt")) {
   fw.write("Hello World");
   fw.write('\n');
   fw.write("New Line");
} catch (IOException e)
 {
 e.printStackTrace();
 }
```

PART 5: BUFFERED STREAMS

1. Why Buffering?

```
┌──────────────────────────────────────────────┐
│              WHY BUFFERING?                  │
├──────────────────────────────────────────────┤
│                                              │
│  Without Buffer:                             │
│  Read 1 byte → System Call → Disk → Memory → Repeat
│  (1000 bytes = 1000 System Calls) → SLOW!    │
│                                              │
│  With Buffer:                                │
│  Read 8192 bytes → System Call → Disk → Memory (Buffer)
│  Serve 8192 bytes from Memory → Fast!        │
│  (1000 bytes = 1 System Call) → FAST!        │
│                                              │
│  Benefit: Reduces expensive OS disk I/O operations.
│                                              │
└──────────────────────────────────────────────┘
```

2. BufferedReader

2.1 What is it?

- **Definition:** Reads text from a character-input stream, buffering characters for efficient reading.
- **Wrapper:** Wraps FileReader or InputStreamReader.

2.2 Key Methods

Method	Description	Returns
read()	Read single char	int
read(char[] cbuf)	Read into array	int
readLine()	**Read whole line**	String (null at EOF)
skip(long n)	Skip characters	long

2.3 Internal Working

- **Buffer Array:** Maintains a char[] cb (default size 8192 chars).
- **Fill Strategy:** When buffer empty, reads large chunk from underlying stream.
- **Line Parsing:** readLine() scans buffer for \n or \r\n.

2.4 How to Use?

```
try (BufferedReader br = new BufferedReader(new FileReader("text.txt"))) {
  String line;
  // Most common pattern for reading text files
  while ((line = br.readLine()) != null) {
    System.out.println(line);
  }} catch (IOException e) { e.printStackTrace(); }
```

3. BufferedWriter

3.1 What is it?

- **Definition:** Writes text to a character-output stream, buffering characters.
- **Wrapper:** Wraps FileWriter or OutputStreamWriter.

3.2 Key Methods

Method	Description
write(String str)	Write string
write(String str, int off, int len)	Write portion
newLine()	Write platform-specific line separator
flush()	**Critical:** Flushes buffer to disk

3.3 Internal Working

- **Buffer Array:** Maintains char[] cb.
- **Write Strategy:** Stores chars in buffer. Only writes to disk when buffer full or flush()/close() called.
- **Risk:** If program crashes before flush(), data in buffer is **lost**.

3.4 How to Use?

java

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"))) {
   bw.write("Line 1");
   bw.newLine();  // Uses \n or \r\n depending on OS
   bw.write("Line 2");
   // flush() called automatically by try-with-resources close()
} catch (IOException e) { e.printStackTrace(); }
```

PART 6: SCANNER CLASS

1. What is Scanner?

- **Definition:** A simple text scanner which can parse primitive types and strings using regular expressions.
- **Package:** java.util
- **Use Case:** Parsing user input (Console), reading structured text files.

2. Why We Use?

- **Convenience:** nextInt(), nextDouble(), nextLine() handles parsing automatically.
- **Delimiters:** Can split input by spaces, commas, etc.
- **Validation:** Throws exception if input doesn't match expected type.

3. Internal Working

- **Buffering:** Uses internal CharBuffer.
- **Pattern Matching:** Uses regex to find tokens (delimiters).
- **Performance: Slower** than BufferedReader for large files due to regex overhead.

4. How to Use?

```
// Console Input
Scanner sc = new Scanner(System.in);
System.out.print("Enter name: ");
String name = sc.nextLine();
System.out.print("Enter age: ");
int age = sc.nextInt();
// File Input
Scanner fileSc = new Scanner(new File("data.txt"));
while (fileSc.hasNext()) {
  String token = fileSc.next();
  System.out.println(token);
}
fileSc.close();
```

5. Common Pitfalls

- **nextInt() followed by nextLine():** nextInt() doesn't consume the newline character. The subsequent nextLine() reads an empty string.
    - **Fix:** Add an extra sc.nextLine() after nextInt().
- **Resource Leak:** Always close Scanner (it closes the underlying stream too).
- **Performance:** Do not use for reading large files (use BufferedReader).

PART 7: SERIALIZATION

1. What is Serialization?

Definition: The process of converting an **Object's state** into a **Byte Stream** to save it to a file, database, or send over network.

2. What is Deserialization?

Definition: The reverse process. Converting a **Byte Stream** back into a **live Object** in memory.

3. Why We Use?

- **Persistence:** Save object state to disk (e.g., game save, user session).
- **Communication:** Send objects over network (RMI, sockets).
- **Caching:** Store complex objects in memory cache (Redis, etc.).

4. How to Use? (The Serializable Interface)

4.1 Marker Interface

java

```
import java.io.*;

// Class MUST implement Serializable

class User implements Serializable {

   String name;

   int age;

   // Transient field (NOT serialized)

   transient String password;

}
```

Serialization code

```java
User user = new User("Alice", 25, "secret123");

try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("user.ser"))) {
    oos.writeObject(user);  // Converts object to bytes
} catch (IOException e) { e.printStackTrace(); }
```

4.3 Deserialization Code

```java
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("user.ser"))) {
    User user = (User) ois.readObject();  // Converts bytes to object
    System.out.println(user.name);  // "Alice"
    System.out.println(user.password); // null (transient)
} catch (IOException | ClassNotFoundException e) { e.printStackTrace(); }
```

5. Internal Working of Serialization

```
┌──────────────────────────────────────────────────────┐
│              SERIALIZATION INTERNAL WORKING           │
├──────────────────────────────────────────────────────┤
│                                                      │
│  1. Check Interface:                                 │
│     JVM checks if class implements java.io.Serializable │
│     If NO → NotSerializableException                 │
│                                                      │
│  2. Write Class Metadata:                            │
│     Writes class name, serialVersionUID, field info  │
│                                                      │
│  3. Write Field Values:                              │
│     Iterates through fields                          │
│     - If primitive: writes value directly            │
│     - If Object: recursively serializes that object  │
│     - If transient: SKIPS field                      │
│     - If static: SKIPS field (belongs to class)      │
│                                                      │
│  4. Handle References:                               │
│     Maintains a handle table to avoid infinite loops │
│     If object referenced twice, writes reference ID  │
│                                                      │
│  5. Deserialization:                                 │
│     - Allocates memory for object (without constructor) │
│     - Populates fields from byte stream              │
│     - Validates serialVersionUID                     │
│                                                      │
└──────────────────────────────────────────────────────┘
```

6. transient Keyword

6.1 What is it?

- **Definition:** A modifier applied to instance variables.
- **Purpose:** Tells the serialization mechanism to **SKIP** this field.

6.2 Why Use?

- **Security:** Don't save passwords, credit card numbers.
- **Performance:** Field is large or can be recalculated (e.g., cachedValue).
- **Compatibility:** Field exists in new version but not old.

6.3 Behavior

- During Serialization: Field is ignored.
- During Deserialization: Field is set to **default value** (null for objects, 0 for ints, false for boolean).

7. serialVersionUID

7.1 What is it?

- **Definition:** A unique identifier (hash code) for a Serializable class.
- **Type:** private static final long serialVersionUID = 1L;

7.2 Why Use?

- **Version Control:** Ensures the sender and receiver of a serialized object have loaded classes for that object that are compatible with respect to serialization.
- **Prevent InvalidClassException:** If you modify a class (add/remove fields), the default UID changes. Deserialization will fail if UIDs don't match.

7.3 Internal Working

7.3 Internal Working

```
┌──────────────────────────────────────────────────┐
│              serialVersionUID VALIDATION          │
├──────────────────────────────────────────────────┤
│                                                  │
│  Scenario 1: UID Matches                         │
│  - File UID: 1L                                  │
│  - Class UID: 1L                                 │
│  - Result: ✅ Deserialization succeeds (fields mapped)  │
│                                                  │
│  Scenario 2: UID Mismatch                        │
│  - File UID: 1L                                  │
│  - Class UID: 2L (Class modified, auto-generated changed)  │
│  - Result: ❌ InvalidClassException              │
│                                                  │
│  Best Practice:                                  │
│  Always declare explicit UID to maintain compatibility  │
│  across code changes.                            │
│                                                  │
└──────────────────────────────────────────────────┘
```

7.4 How to Declare?

```java
class User implements Serializable {
    private static final long serialVersionUID = 123456789L;
    // ... fields
}
```

COMPLETE COMPARISON & METHODS

1. Stream Comparison Table

Stream Class	Type	Data Unit	Use Case	Performance
FileInputStream	Byte	8-bit Byte	Images, Audio, Binary	Fast (Raw)
FileOutputStream	Byte	8-bit Byte	Images, Audio, Binary	Fast (Raw)
FileReader	Character	16-bit Char	Text Files	Medium (Encoding)
FileWriter	Character	16-bit Char	Text Files	Medium (Encoding)
BufferedReader	Character	16-bit Char	Text (Line by line)	**Fastest for Text**

Stream Class	Type	Data Unit	Use Case	Performance
BufferedWriter	Character	16-bit Char	Text (Writing)	**Fastest for Text**
ObjectInputStream	Byte	Object	Deserialization	Slow (Reflection)
ObjectOutputStream	Byte	Object	Serialization	Slow (Reflection)
Scanner	Character	Tokens	Console Input, Parsing	Slow (Regex)

2. Important Methods Reference

Class	Method	Description
InputStream	read()	Reads byte (blocks)
	close()	Releases resources
OutputStream	write(int b)	Writes byte
	flush()	Forces write (buffered)
Reader	read()	Reads char
	read(char[] cbuf)	Reads into array
Writer	write(String str)	Writes string
	append(CharSequence)	Appends text
BufferedReader	readLine()	Reads full line (very useful)
BufferedWriter	newLine()	Writes OS-specific line separator
ObjectOutputStream	writeObject(Object obj)	Serializes object
ObjectInputStream	readObject()	Deserializes object
Scanner	nextInt(), nextLine()	Parses tokens
	hasNext()	Checks for more tokens
File	createNewFile()	Creates physical file
	delete()	Deletes file
	exists()	Checks existence

INTERNAL WORKING SUMMARY

1. OS Interaction

- Java I/O classes are wrappers around **OS System Calls**.
- FileInputStream → open() → Returns **File Descriptor (FD)**.
- Read/Write operations use FD to interact with Kernel Buffer.
- **Blocking I/O:** Thread waits until OS completes the operation.

2. Buffering Mechanism

- **Memory Array:** BufferedInputStream allocates a byte[] buffer (default 8192 bytes).
- **Fill:** When read() is called, if buffer empty, it reads 8192 bytes from OS in **one call**.
- **Serve:** Subsequent read() calls return data from memory array (fast).
- **Flush:** BufferedOutputStream stores data in array. Only writes to OS when full or flush() called.

3. Serialization Graph

- **Object Graph:** Serialization doesn't just save one object; it saves all objects referenced by that object (recursively).
- **Cycle Detection:** Uses a handle table (HandleTable) to track objects already written. If an object is encountered again, it writes a reference ID instead of duplicating data.
- **Constructor Bypass:** Deserialization uses Unsafe.allocateInstance() (or similar) to create object **without calling constructor**. This is why static initializers run, but instance constructors do not.

PART 9: BEST PRACTICES & PITFALLS

1. Try-With-Resources (Critical)

Always close streams to prevent resource leaks (file handles).

```java
// ✅ GOOD (Java 7+)
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    // use br
} // Auto-closes here

// ❌ BAD (Old way)
BufferedReader br = new BufferedReader(new FileReader("file.txt"));
// ... use br
br.close(); // If exception occurs before this, resource leaks!
```

2. Character Encoding

`FileReader` uses default charset. This breaks on different OS.

```java
// ❌ Risky
new FileReader("file.txt");

// ✅ Safe (Specify Charset)
new InputStreamReader(new FileInputStream("file.txt"), StandardCharsets.UTF_8);
// Or use Java 7+ Files class
Files.newBufferedReader(Paths.get("file.txt"), StandardCharsets.UTF_8);
```

3. Serialization Security

Never deserialize untrusted data. It can lead to Remote Code Execution (RCE).

```java
// ❌ DANGEROUS
ObjectInputStream ois = new ObjectInputStream(socket.getInputStream());
User u = (User) ois.readObject(); // Attacker can send malicious object

// ✅ SAFE
// Validate object class before deserializing
// Or use JSON/XML instead of Java Serialization
```

4. Buffering

Always wrap file streams with Buffered streams for performance.

```java
// ❌ Slow (Many system calls)
new FileReader("file.txt");

// ✅ Fast (Fewer system calls)
new BufferedReader(new FileReader("file.txt"));
```

5. Static & Transient

- **Static:** Never serialized (belongs to class, not object).
- **Transient:** Never serialized (skipped during save).
- **Constructor:** Not called during deserialization (object is reconstructed from bytes).

. Introduction to Multithreading

Theory: Multithreading is a computational paradigm that allows a single process to manage multiple execution flows concurrently. By dividing a program into independent threads, the CPU can switch between tasks rapidly, creating the illusion of parallelism even on single-core systems. This model maximizes CPU utilization by keeping the processor busy during I/O waits and improves application responsiveness by separating long-running tasks from the main user interface.

1.1 What is it?

- **Definition:** The ability of a CPU to execute multiple threads of execution concurrently within a single process.
- **Core Concept:** A thread is the **smallest unit of execution** that can be scheduled by an operating system.
- **Goal:** Achieve concurrency (dealing with lots of things at once) and parallelism (doing lots of things at once).

1.2 Why We Use?

- **Responsiveness:** Keeps UI active while background tasks run (e.g., downloading while scrolling).
- **Resource Utilization:** Uses CPU time during I/O waits (disk/network) by switching to another thread.
- **Economy:** Threads share process memory, making them cheaper to create than processes.
- **Simplification:** Breaks complex problems into smaller, independent sequential tasks.

1.3 Internal Working

1.3 Internal Working

```
┌──────────────────────────────────────────────────┐
│                MULTITHREADING ARCHITECTURE                │
├──────────────────────────────────────────────────┤
│                                                  │
│  Process (JVM)                                   │
│  ┌──────────────────────────────────────────┐    │
│  │  Shared Memory (Heap, Code, Data)        │    │
│  │  ┌──────────┐  ┌──────────┐  ┌──────────┐    │
│  │  │ Thread 1 │  │ Thread 2 │  │ Thread 3 │    │
│  │  │ (Stack)  │  │ (Stack)  │  │ (Stack)  │    │
│  │  └──────────┘  └──────────┘  └──────────┘    │
│  └──────────────────────────────────────────┘    │
│                                                  │
│  Key Point: Threads share Heap, but have private Stacks.  │
│                                                  │
└──────────────────────────────────────────────────┘
```

2. Process vs Thread

Theory: A process is an independent program in execution with its own memory space, while a thread is a lightweight subset of a process that shares the process's resources. Processes are isolated from each other for security and stability, whereas threads within the same process can communicate directly through shared memory. This fundamental difference dictates how context switching occurs, with threads being significantly faster to switch than processes due to lower overhead.

2.1 Key Differences

Feature	Process	Thread
Memory	Separate address space (isolated)	Shared address space (within process)
Communication	Inter-Process Communication (IPC)	Direct (via shared variables)
Creation Cost	High (OS kernel involvement)	Low (shares process resources)
Context Switch	Slow (save/restore full state)	Fast (save/restore registers/stack)
Crash Impact	One process crash doesn't affect others	One thread crash can kill entire process
Overhead	Heavy (PCB, memory tables)	Light (TCB, stack pointer)

2.2 Internal Working

- **Process Control Block (PCB):** OS maintains a PCB for each process (memory limits, open files).
- **Thread Control Block (TCB):** OS maintains a TCB for each thread (program counter, registers, stack pointer).
- **Context Switch:** When switching threads, the OS only updates the TCB. When switching processes, it must update memory maps (TLB flush), which is expensive.

2.3 When to Use Which?

- **Use Processes:** When isolation is critical (e.g., browser tabs, microservices).
- **Use Threads:** When high performance and shared data access are needed (e.g., web server handling requests).

---

3. Multitasking

Theory: Multitasking is the operating system's ability to execute multiple tasks (processes or threads) concurrently by rapidly switching the CPU between them. It creates the illusion that multiple programs are running simultaneously, even on a single-core processor. This is achieved through a scheduler that allocates time slices to each task, ensuring fair distribution of CPU resources and preventing any single task from monopolizing the system.

3.1 Types of Multitasking

Type	Description	Example
Process-Based	Running multiple programs concurrently	Running Chrome + Word + Spotify
Thread-Based	Running multiple threads within one program	Web server handling multiple requests

3.2 Internal Working

- **Scheduler:** OS component that decides which thread runs next.
- **Time Quantum:** Fixed time slice allocated to each thread (e.g., 10ms).
- **Context Switch:** Saving state of current thread, loading state of next thread.

3.3 Why We Use?

- **Efficiency:** Keeps CPU busy when one task waits for I/O.
- **User Experience:** Allows users to interact with one app while another works in background.
- **Throughput:** Increases total amount of work completed in a given time.

Multi-tasking

Executing multiple tasks simultaneously is the concept of multitasking
example: classroom student multi-tasking like listening the class, taking the notes, observing, talking to the friends

This multitasking divided into two categories
1. Process based multi-tasking
2. Thread based multi-tasking

Process-based multi-tasking:

"Executing several tasks simultaneously, where each task separates independent program(process) is called process-based multitasking". there is no dependency between first process and second process and there is no dependency between second and third process executing several tasks simultaneously where each task is separate independent program such type of multitasking is nothing but process-based multitasking. It is best suitable for operating system level (os level) not for programmatic level

Simple example:

when the programmer doing the java program and listening audio songs from same system and at a time download file from the google these above tasks doing simultaneously where each task separate independent process there is no dependency such type of multitasking is called process-based multitasking

Listening to music (one process)

Browsing the internet (second process)

Downloading a file (third process)

Thread-Based Multi-Tasking:

"Executing several tasks simultaneously where each task is separate independent part of same program such type of multitasking is called Thread based Multitasking and each independent part is called thread". Thread based multi -tasking is best suitable at programmatic level . simply this means multiple parts of the same program run at the same time.

Example: A Java application can perform:

- Saving a file
- Performing calculations
- Refreshing UI

... all at once, using *threads*.

✓ Lightweight
✓ Fast to create
✓ Threads share memory of the same process

Main advantage of multi-tasking

- whether it is process based or thread based the main objective of multi-tasking is o reduce response time of the system and to improve performance
  - It gives better performance
  - Efficient CPU usage
  - Prevents application from freezing
  - Ideal for real-time tasks
  - Easy to implement

where we used the multi -tasking system

Multitasking = running multiple things at the same time

We use it in:

- Operating systems (process multitasking)

- Java applications (thread multitasking)
- Web servers
- Background tasks
- Mobile apps,
- Games
- Data processing
- Network apps

Anywhere you need **speed + responsiveness + parallel execution**, you use multitasking.

Where we can we use multi-threading

- Multimedia Graphics
- Animations
- To develop video games
- To develop web servers and application servers etc

Difference: Process Multitasking vs Thread Multitasking

Feature	Process	Thread
Meaning	Multiple programs running	Multiple tasks in the same program
Memory	Each has its own memory	Threads share memory
Speed	Slower	Faster
Communication	Harder	Easier (shared data)
Java Example	Running two Java apps	Multithreading inside one app

When comparing old languages developing multi-threaded applications is easy, java provides inbuilt support in multithreading with rich API [Thread,Runnable,ThreadGroup...etc].

What is Thread

Thread is a separate flow of execution. for every flow or every thread of execution is separate job is there. A Java thread is the smallest unit of execution within a program. It is a lightweight subprocess that runs independently but shares the same memory space as the process, allowing multiple tasks to execute concurrently.

4. Thread Creation

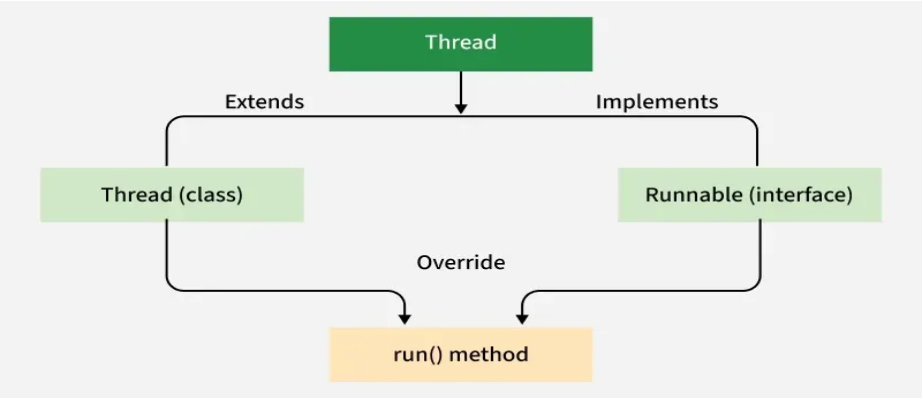
Theory: Creating a thread in Java involves instantiating a Thread object and defining the code it should execute, typically by overriding the run() method. The JVM maps this Java thread to a native OS thread using platform-specific APIs like pthread_create on Linux or CreateThread on Windows. This abstraction allows Java developers to write concurrent code without managing low-level OS thread details, though understanding the underlying mapping is crucial for performance tuning.

Defining a Threads in Java

We can define a thread in java using two ways

1.By extending thread class

2.By implementing a Runnable Interface



Big Picture model

Your Java code

↓

Thread.start()

↓

Native JVM code (C/C++)

↓

Operating System Thread

↓

CPU Scheduler

↓

run() executes

By extending thread class:

Create a class that extends Thread. Override the run() method, this is where you put the code that the thread should execute. Then create an object of your class and call the start() method. This will internally call run() in a new thread. **What:** Inherit from java.lang.Thread and override run(). **Why:** Simple implementation for small tasks. **How:**

Example:

```
Class Mythread extends Thread
{
public void run()

{

for(int i=0;i<10;i++)

{

System.out.println("child thread");//executed by child thread
}

}

}


Class ThreadDemo

{

public static void main(String[] args)

{

//main thread is responsible for to execute the main method, main method not itself main
thread

Mythread k=new Mythread();  // Thread Instantiation main thread

k.start(); // starting of thread // child thread -> main thread starts child thread

for(int i=0;i<10;i++)

{

System.out.println("main Thread"); //main thread
}

}
```

}

Internal: start() calls native start0() which allocates OS stack and registers.

Possible outputs,

Output-1	output-2	output-3
main thread -10,	child thread-10	child thread-5 main thread-5
Child thread-10,	main thread-10 M	main thread-5 child 5

Case 1: Thread scheduler

Scheduling threads

it is part of jvm it is responsible to schedule threads i.e. if multiple threads are waiting t get the chance of execution than in which order threads will be executed is decided by thread scheduler. We cannot except exact algorithm by thread scheduler it is varied from jvm to jvm .hence we cannot expect thread execution order and thread exact output hence whenever situation comes to multi-threading there is no guarantee for exact output but we can we can provide several possible outputs

case2: t.start and t.run

Why run() alone doesn't create a thread

t.run(); // This is just a **normal method call**.

t.start(); //

This:

- Creates a new OS thread
- Registers it with JVM
- Calls run() *inside that new thread*

t.start(); In the case of t.start() a new thread will be created which is responsible for the execution of run method

t.run(): in the case of t.run() a new thread won't be created and the run method will be executed just like run method call by main thread .

hence in the above program if we replace t.start() with t.run() then the output is like normal execution this total output produced by only main thread.

Case 3: importance thread class start()

Threadclass.start() method is responsible to register the thread with thread scheduler and all other mandatory activities hence without executing thread class start method there is no chance of starting a new thread in java due to this thread class start method is considered as heart of multi threading

Why we call start() method

Start() :

{

1. Register this thread with Thread Scheduler
2. Perform all other mandatory activities
3. invoke run();

}

Case 4: overloading of run() method

Overloading of run method is always possible but threadclass.start() can invoke no argument run() the other overloaded method we have to call explicitly like a normal method call

Example

```
package org.atyeti.multiThreading.threadclass;

public class Mythread1 extends Thread{
  public void run()
  {
    System.out.println("no args ");
  }
  public void run(int i)
  {
    System.out.println("args run");
  }

  public static void main(String[] args) {
    Mythread1 my=new Mythread1();
    my.start();
  }
}
```

op; no args

case 5: empty class

if you are not overriding run method then thread class run method will be executed which has empty implementation. Hence we won't get any output

```
class Mythread extends Thread

{

}

Class Test

{

Public static void main(string [] args)

{

Mythread t=new My thread();

t.start();

}

}
```

It is highly recommended to override run method otherwise don't go for the multi threading concept.

Case 6: overriding of start() method

It not recommended to override a start method otherwise don't go for the multi threading concept

4.2 Implementing Runnable Interface

What: Implement java.lang.Runnable and pass to Thread constructor. **Why:** Preferred approach; allows extending other classes; separates task from thread management.

How:

```
class MyTask implements Runnable {

   public void run() { System.out.println("Task"); }

}

Thread t = new Thread(new MyTask());

t.start();
```

Internal: Thread.run() checks if target (Runnable) is not null and calls target.run().

4.3 Lambda Thread Creation

What: Use Java 8 lambda expression to implement Runnable inline.

Why: Concise syntax; reduces boilerplate code for simple tasks.

How:

```
Thread t = new Thread(() -> System.out.println("Lambda"));

t.start();
```

Dameon threads

A daemon thread is a low-priority background thread that does not prevent the JVM from exiting when the program finishes execution

- Daemon threads run in the background to support user threads.
- The JVM exits automatically when all user (non-daemon) threads complete.
- They are created using the same Thread class but are marked as daemon using the setDaemon(true) method.
- The setDaemon(true) method must be called before the thread starts.
- Common examples include Garbage Collector and Finalizer Thread.

Key Characteristics and Usage

- **Purpose:** Used for background supporting tasks like monitoring, garbage collection, or automatic saving.
- **Lifecycle:**

The JVM does not wait for daemon threads to finish execution; they are killed immediately when the application exits.

- **Creation:** A thread is marked as a daemon using setDaemon(true) before it is started.
- **Inheritance:** A child thread created by a daemon thread is automatically a daemon thread.
- **Restriction:** Calling setDaemon(true) after a thread has started throws an IllegalThreadStateException.

Examples

- **Garbage Collector (GC):** A system-defined daemon thread that runs in the background.
- **Finalizer:** A thread that runs finalization methods.
- **Application Services:** Background tasks like saving files, monitoring services, or cleaning up resources.

Syntax: thread.setDaemon(true);

Daemon vs. User Threads

Feature	Daemon Thread	User Thread
Priority	Low	High (usually)
JVM Shutdown	Does not prevent exit	JVM waits for completion
Usage	Background services	Main application tasks

Important Considerations

- If a daemon thread is used for I/O operations, it might be terminated abruptly, leaving files or resources in an inconsistent state.
- They are best suited for tasks that do not require finalization, such as background monitoring or data cleanup.

Methods Used

- **void setDaemon(boolean on):** Marks a thread as daemon or user thread. Must be called before start().
- **boolean isDaemon():** Checks whether a thread is daemon.

Example :

```
public class DaemonExample extends Thread {
  public void run() {
    if (Thread.currentThread().isDaemon()) {
      System.out.println("Daemon thread running...");
    } else {
      System.out.println("User thread running...");
    }
  }
  public static void main(String[] args) {
    DaemonExample t1 = new DaemonExample();
    DaemonExample t2 = new DaemonExample();
    t1.setDaemon(true);  // must be set before start()
    t1.start();
    t2.start();
  }
}
```

important Notes

- A thread inherits the daemon status of the thread that creates it.
- The setDaemon(true) method throws IllegalThreadStateException if called after start().
- Daemon threads should not be used for tasks requiring completion, such as writing to a file or updating a database.
- JVM terminates all daemon threads abruptly without performing cleanup operations.

5️⃣ Advantages of Multithreading

✅ Better CPU utilization
✅ Responsiveness (UI stays alive)
✅ Faster I/O operations
✅ Resource sharing

6️⃣ Disadvantages

❌ Race conditions
❌ Deadlocks
❌ Hard to debug
❌ Context switching overhead

7️⃣ Real-world Examples

- Web server handling multiple clients
- UI thread + background worker
- File downloader (parallel chunks)

8️⃣ When NOT to Use Multithreading

❌ Heavy shared state
❌ Simple linear tasks
❌ CPU-bound on single core
❌ When correctness > speed

🟢 STAGE 2 – THREAD CREATION & LIFECYCLE

(Safe learning zone)

1️⃣ Thread Life Cycle

States:

1. **New** – Created, not started
2. **Runnable** – Ready to run
3. **Running** – Executing
4. **Waiting / Blocked** – Sleeping or waiting
5. **Dead** – Finished

⚠️ Runnable ≠ Running

Thread Life Cycle in Java

A thread goes through these states (as per java.lang.Thread.State):

- **NEW**: Thread created but not yet started (start() not called).
- **RUNNABLE**: Thread is ready to run—it may be **waiting for CPU time** or **currently executing**.
  → *Note:* In Java, "Runnable" includes both **ready** and **running** states (unlike some OS models that separate them).
- **Running** : Executing instructions and triggers the os scheduler picks it
- **BLOCKED**: Waiting to acquire a monitor lock (e.g., entering a synchronized block).
- **WAITING**: Waiting indefinitely for another thread (e.g., wait(), join(), or LockSupport.park()).
- **TIMED_WAITING**: Waiting for a specified time (e.g., sleep(1000), wait(5000)).
- **TERMINATED**: Thread has finished execution.

5.2 Internal Working

```
┌──────────────────────────────────────────────────────┐
│                  STATE TRANSITIONS                   │
├──────────────────────────────────────────────────────┤
│                                                      │
│  NEW ──start()──> RUNNABLE ──schedule──> RUNNING     │
│                        ^       │                     │
│                        │       ├──sleep()──> TIMED_WAITING│
│                        │       ├──wait()───> WAITING │
│                        │       └──lock fail-> BLOCKED │
│                        │                             │
│  RUNNING ──done────────> TERMINATED                  │
│                                                      │
└──────────────────────────────────────────────────────┘
```

- RUNNABLE vs RUNNING: Java combines these. `Thread.getState()` returns `RUNNABLE` for both waiting for CPU and actually running.
- BLOCKED: Specific to `synchronized` locks.
- WAITING: Specific to `Object.wait()` or `Thread.join()`.

5.3 How to Check

```java
Thread t = new Thread(() -> {});
System.out.println(t.getState()); // NEW
t.start();
System.out.println(t.getState()); // RUNNABLE
```

2️⃣ start() vs run()

run()

- Normal method call
- Runs in **current thread**

- **What:** Entry point for thread logic.
- **Internal:** If `target` (Runnable) exists, calls `target.run()`, else does nothing.
- **Rule:** Do NOT call directly (runs in current thread). Override this to define task.
- **Why:** Defines the code block that executes concurrently.
-

start()

- Creates **new thread**
- Calls run() internally

- **What:** Initiates thread execution.
- **Internal:** Calls native `start0()` → OS creates thread → JVM calls `run()`.
- **Rule:** Can only be called once. `IllegalThreadStateException` if called twice.
- **Why:** Separates thread creation from execution; allows JVM to setup stack.
-

📌 start() → OS involvement

📌 run() → no threading

Method	Behavior
run()	Just a regular method call. Executes in the **current thread**. No new thread is created.
start()	**Creates a new thread**, registers it with the JVM and OS, and **calls run() in that new thread**.

✅ Always use start() to launch a new thread. Calling run() directly defeats the purpose of multithreading.

3️⃣ Thread Scheduler

- OS decides:
  - Which thread runs
  - When
  - For how long

Not predictable
Not controllable

- Managed by the **OS** (not Java itself).
- Decides:
  - Which RUNNABLE thread gets CPU time
  - When to switch (preemption)
  - Time slice duration
- **Non-deterministic**: You cannot predict or control exact scheduling.
- Use Thread.yield(), sleep(), or concurrency utilities (like CountDownLatch) for coordination—not assumptions about scheduling.

4️⃣ Naming Threads

- For debugging
- Helps in logs and stack traces
- Set name via constructor or setName()
- `Thread t = new Thread(runnable, "Worker-1");`
- `// or`
- `t.setName("IO-Thread");`
-

Example:

Worker-1

IO-Thread

GC-Thread

6️⃣ Daemon Threads

- Background threads
- JVM exits when only daemon threads remain

Examples:

- Garbage collector

- Background cleanup tasks

⚠️ Daemon threads may be killed abruptly

7️⃣ Joining Threads

- Make one thread wait for another to finish.
- Guarantees that a background task completes before the main program proceeds (e.g., before printing results or shutting down).

6.4 join()

- **What:** Waits for another thread to die.
- **Internal:** Calls `wait()` on the thread object internally until state is `TERMINATED`.
- **Rule:** Current thread blocks. Throws `InterruptedException`.
- **Why:** Ensure dependency order (e.g., wait for initialization before processing).
-

Why it matters

Imagine you start a thread to download a file, but your main program exits immediately—the download never finishes. join() prevents that.

- One thread waits for another to finish

main → waits for worker

Used to:

- Ensure task completion
- Avoid premature exit

- `thread.join()` → current thread **waits** until `thread` finishes.
- Useful for **synchronization** between threads.
- Overloads:
  - `join()` → wait forever
  - `join(long millis)` → wait up to specified time

Example:

- Thread worker = new Thread(() -> { /* task */ });
- worker.start();
- worker.join(); // main thread waits here
- System.out.println("Worker done!");

Example

```
Thread downloader = new Thread(() -> {
  System.out.println("Downloading...");
  try { Thread.sleep(3000); } catch (InterruptedException e) {}
  System.out.println("Done!");
});
downloader.start();
downloader.join(); // Main thread BLOCKS here until 'downloader' finishes
System.out.println("Main continues — safe to exit now.");
```

✅ **Key point**: Without join(), the main thread might exit while background work is still running → **incomplete work, lost data**.

📌 Use join() when you **depend on the result** or **must ensure cleanup**.

8️⃣ Sleeping Threads

- sleep(time)
- Thread pauses for fixed duration
- Does **not release locks**

- **What:** Pauses current thread execution.
- **Internal:** Sets thread state to `TIMED_WAITING`; registers timer with OS.
- **Rule:** Does NOT release locks. Throws `InterruptedException`.
- **Why:** Simulate delay, prevent CPU hogging, pace operations.
-

<u>What it does</u>

- Pauses the **current thread** for a specified time (in milliseconds).
- **Does NOT release locks** (unlike wait()).
- **Common uses**
- Introducing delays (e.g., retry after 1 second)
- Simple polling (e.g., "check every 500ms if job is done")

- ⚠️ **But**: Polling is **inefficient**—better alternatives exist (like wait/notify or CompletableFuture).

  Example

  ```
  synchronized (lock) {
    Thread.sleep(1000); // Still holds 'lock' during sleep!
  }
  ```

  Never use sleep() as a synchronization tool—it doesn't coordinate with other threads.

Used for:

- Delays
- Polling (not ideal)

🟡 STAGE 3 – THREAD CONTROL & SIGNALING

(Still safe, no shared data yet)

1️⃣ yield()

What it does

- Hints to the scheduler: *"I'm not in a hurry—let another RUNNABLE thread run."*
- **No guarantee** anything will happen.
- Rarely affects modern OS schedulers (they're already smart).

yield()

- **What:** Hints to scheduler to pause current thread.
- **Internal:** Moves thread from `RUNNING` to `RUNNABLE`.
- **Rule:** No guarantee another thread will run.
- **Why:** Polite behavior; allow other threads of same priority to run.
-

Reality check

- Almost **never needed** in real code.
- Don't rely on it for correctness or performance.

✅ Think of it as saying "after you!" in a doorway—but the other person might ignore you.

- Suggests scheduler:

"I'm okay to pause"

- No guarantee
- Rarely useful

2️⃣ interrupt()

Crucial insight

- Java does NOT support forceful thread termination (for safety!).
- Instead, it uses cooperative interruption:
  → One thread requests another to stop.
  → The target thread decides when and how to stop.
- What: Requests thread to stop.
- Internal: Sets interrupt flag. If blocked (sleep/wait), throws InterruptedException.
- Rule: Cooperative cancellation (thread must check flag).
- Why: Graceful shutdown mechanism; breaking out of infinite loops.

How it works

- Calling thread.interrupt() sets an internal boolean flag (the *interrupt status*) on that thread.
- If the thread is blocked in sleep(), wait(), or join(), it immediately throws InterruptedException.
- Sends **interrupt signal** to thread
- Does NOT stop thread forcibly
- Cooperative mechanism

  Example:

```
Thread worker = new Thread(() -> {
  while (!Thread.currentThread().isInterrupted()) {
    doSomeWork(); // Keep working unless asked to stop
  }
  System.out.println("Worker shutting down gracefully.");
```

```
});
worker.start();
// Later...
worker.interrupt(); // Sends signal: "Please stop when convenient"
```

3️⃣ isInterrupted()

- Checks interrupt flag
- Does not clear it

- Returns `true` if the thread has been interrupted.
- **Does NOT reset** the interrupt flag.
-

Used for:

```
while (!isInterrupted()) { work }
```

4️⃣ InterruptedException (Correct Handling)

Thrown when thread is:

- Sleeping
- Waiting
- Joining

Correct pattern:

- Catch
- Restore interrupt flag
- Exit gracefully

📌 Never swallow interrupts silently

```
try {
  Thread.sleep(1000);
} catch (InterruptedException e) {
  // Step 1: Restore the interrupt flag
  Thread.currentThread().interrupt();
```

```
  // Step 2: Exit the method or loop
  return; // or break;
}
```

5️⃣ Graceful Thread Shutdown

❌ stop() (unsafe, deprecated)
✅ Use:

- Interrupt flag
- Volatile boolean

Thread checks signal and exits cleanly.

- Deprecated since Java 1.2!
- Cause **data corruption**, **deadlocks**, and **inconsistent state**.

✅ **Safe approaches:**

Option A: Interruption (Preferred)

Best for threads that **block** or **check interruption regularly**.

```
Thread worker = new Thread(() -> {
  while (!Thread.currentThread().isInterrupted()) {
    try {
      doWork();
      Thread.sleep(100); // blocking → responsive to interrupt
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt(); // restore
      break; // exit
    }
  }
});
```

Option B: Volatile boolean flag

Use when you **can't use interruption** (e.g., inside a library that swallows interrupts).

```
private volatile boolean running = true;


Thread worker = new Thread(() -> {
  while (running) {
    doWork();
  }
});


// To stop:
running = false;
```

Final Mental Model (Lock this in)

OS schedules **threads**, not code
Threads share **memory**, not safety
Interrupts are **requests**, not commands
Scheduling is **non-deterministic**

-
- **Threads are cooperative citizens**, not slaves.
- **Never force-stop**—always **ask politely** (interrupt()).
- **Always respond** to interruption signals.
- **Graceful shutdown = reliability + user trust**.

6.7 isAlive()

- **What:** Checks if thread is started and not terminated.
- **Internal:** Checks internal threadStatus flag.
- **Rule:** Returns false for NEW and TERMINATED.
- **Why:** Monitor thread status without joining.

6.8 setPriority() / getPriority()

- **What:** Sets/gets thread priority (1-10).
- **Internal:** Maps to OS priority (e.g., Nice value on Linux).
- **Rule:** MIN_PRIORITY (1), NORM_PRIORITY (5), MAX_PRIORITY (10).
- **Why:** Hint scheduler to give more CPU time to critical threads.

6.9 setName() / getName()

- **What:** Sets/gets thread name.
- **Internal:** Stores string in Thread object.
- **Rule:** Default names: Thread-0, Thread-1.
- **Why:** Debugging (stack traces), logging, monitoring.

8. Thread Scheduling

Theory: Thread scheduling is the algorithm used by the JVM and Operating System to decide which runnable thread gets access to the CPU and for how long. Java relies on the underlying OS scheduler, meaning behavior can vary across platforms (Windows vs. Linux vs. macOS). The scheduler uses priorities and time-slicing to ensure fairness, though Java provides APIs to hint at preferences which the OS may or may not honor.

8.1 Thread Priority

- **What:** Integer value (1-10) indicating relative importance.
- **Internal:** Mapped to OS-specific priority levels (e.g., Windows thread priority classes).
- **Rule:** Higher priority threads are *more likely* to run, not guaranteed.
- **Why:** Optimize performance for critical tasks (e.g., UI thread vs. background log).

8.2 Preemptive Scheduling

- **What:** OS can interrupt a running thread to give CPU to another.
- **Internal:** Timer interrupt triggers context switch.
- **Rule:** Default in all modern Java JVMs.
- **Why:** Prevents one thread from hogging the CPU indefinitely.

8.3 Time Slicing

- **What:** Dividing CPU time into small chunks (quanta) for each thread.
- **Internal:** OS scheduler assigns fixed time (e.g., 10-100ms) per thread.

- **Rule:** After time slice expires, thread is preempted and moved to back of queue.
- **Why:** Ensures fair sharing of CPU among all runnable threads.

8.4 Internal Working

```
┌──────────────────────────────────────────────────┐
│                  SCHEDULER QUEUE                 │
├──────────────────────────────────────────────────┤
│                                                  │
│  Ready Queue: [Thread-1 (Prio 10), Thread-2 (Prio 5), ...] │
│                                                  │
│  Algorithm:                                      │
│  1. Select highest priority thread               │
│  2. If tie, use Round-Robin (Time Slicing)       │
│  3. Run for Time Quantum                         │
│  4. If not done, move to back of queue           │
│                                                  │
│  Note: Java priorities are hints. OS may ignore them.     │
│                                                  │
└──────────────────────────────────────────────────┘
```

This module covers the critical mechanisms for ensuring thread safety in Java. We will explore synchronization theory, intrinsic locks, memory visibility, and atomic operations with internal JVM workings.

---

1. Introduction to Synchronization

Theory: Synchronization is the mechanism that controls access to shared resources by multiple threads to prevent data inconsistency. It ensures that only one thread can execute a critical section of code at a time, enforcing mutual exclusion. Beyond exclusivity, synchronization also guarantees visibility, ensuring that changes made by one thread are immediately visible to other threads.

1.1 What is it?

- Definition: The process of coordinating concurrent threads to access shared resources safely.
- Core Goal: Prevent Race Conditions and ensure Data Consistency.
- Mechanism: Uses Locks (Monitors) to control thread access.

1.2 Why We Use?

- Thread Safety: Prevents data corruption when multiple threads modify shared state.
- Visibility: Ensures updates to variables are seen by other threads (solves caching issues).
- Atomicity: Ensures compound operations (read-modify-write) happen indivisibly.
- Ordering: Prevents instruction reordering that could break logic.\

1.3 Internal Working

```
┌──────────────────────────────────────────────┐
│              SYNCHRONIZATION GOALS            │
├──────────────────────────────────────────────┤
│                                              │
│  1. Mutual Exclusion (Atomicity)             │
│     ┌──────────┐                             │
│     │ Thread 1 │ ──→ Acquires Lock → Executes → Releases │
│     └──────────┘                             │
│                                              │
│     ┌──────────┐                             │
│     │ Thread 2 │ ──→ Waits → Acquires → Executes │
│     └──────────┘                             │
│                                              │
│  2. Visibility (Memory Consistency)          │
│     Thread 1: Write to Var → Flush to Main Memory │
│     Thread 2: Read from Main Memory → See updated value │
│                                              │
└──────────────────────────────────────────────┘
```

3. Synchronization Mechanisms

3.1 Intrinsic Lock (Monitor)

Theory: Every Java object has an intrinsic lock (also called a monitor lock) associated with it. When a thread acquires the lock, it enters the monitor; when it releases the lock, it exits the monitor. Only one thread can hold the lock for a specific object at any given time.

Definition: A built-in locking mechanism associated with every Java object.

Internal Working (Object Header):

Internal Working (Object Header):

```
┌──────────────────────────────────────────────┐
│            OBJECT HEADER (MARK WORD)          │
├──────────────────────────────────────────────┤
│                                              │
│  Every Object in JVM has a Header:           │
│  ┌──────────────────────────────────────┐    │
│  │  Mark Word (64-bit)                  │    │
│  │  [Identity Hashcode | Age | Lock Flags | ... ]  │    │
│  └──────────────────────────────────────┘    │
│                                              │
│  Lock Flags indicate state:                  │
│  001: No Lock                                │
│  010: Lightweight Lock (CAS)                 │
│  100: Heavyweight Lock (OS Mutex)            │
│  101: Biased Lock (Single thread)            │
│                                              │
│  When synchronized is used:                  │
│  1. Thread tries to update Mark Word via CAS │
│  2. If success: Thread owns lock             │
│  3. If fail: Thread blocks (OS Mutex)        │
│                                              │
└──────────────────────────────────────────────┘
```

Properties:

- **Mutual Exclusion:** Only one holder.
- **Reentrancy:** Same thread can acquire lock it already holds.
- **Automatic Release:** Released when method/block exits.

3.2 Synchronized Method

Theory: Applying the synchronized modifier to a method ensures that only one thread can execute that method on a specific instance (or class) at a time. It locks the entire method, which is convenient but can be coarse-grained, potentially reducing concurrency.

Definition: A method that acquires the intrinsic lock of the object (or class) before execution.

Types:

1. Instance Method: Locks this (current object).
2. Static Method: Locks the Class object.

Internal Working (Bytecode)**:**

Copy | Ask Qwen | Explain | Translate(en-US)

```java
public synchronized void method() { }
```

Compiled Bytecode:

```
┌──────────────────────────────────────────────────┐
│              SYNCHRONIZED METHOD BYTECODE         |
├──────────────────────────────────────────────────┤
│                                                  │
│  Access Flags: ACC_SYNCHRONIZED                  │
│                                                  │
│  Execution:                                      │
│  1. Invoke method                                │
│  2. JVM sees ACC_SYNCHRONIZED flag               │
│  3. Executes monitorenter (lock)                 │
│  4. Execute method body                          │
│  5. Executes monitorexit (unlock)                │
│                                                  │
│  Note: Lock is on 'this' for instance, 'Class' for static.  |
│                                                  │
└──────────────────────────────────────────────────┘
```

When to Use:

- ✅ When entire method needs protection.
- ✅ Simple implementation.
- ❌ Avoid if only a small part needs locking (performance).

3.3 Synchronized Block

Theory: A synchronized block allows you to specify the object whose lock should be acquired, providing finer-grained control than synchronized methods. This reduces contention by locking only the critical section of code rather than the entire method.

Definition: A block of code protected by a lock on a specified object.

Syntax: synchronized(objectReference) { // critical section }

Internal Working:

```java
synchronized(lockObj) {
    // code
}
```

Compiled Bytecode:

```
┌──────────────────────────────────────────────────┐
│              SYNCHRONIZED BLOCK BYTECODE          │
├──────────────────────────────────────────────────┤
│                                                  │
│  0: aload_1        (Load lockObj)                │
│  1: dup                                          │
│  2: astore_2                                     │
│  3: monitorenter   (ACQUIRE LOCK)                │
│  4: ...            (Code body)                   │
│  5: aload_2                                      │
│  6: monitorexit    (RELEASE LOCK)                │
│  7: goto           (Normal exit)                 │
│  8: astore_3       (Exception handler)           │
│  9: aload_2                                      │
│ 10: monitorexit    (RELEASE LOCK on Exception)   │
│                                                  │
│  Key: monitorexit is guaranteed even on exception.│
│                                                  │
└──────────────────────────────────────────────────┘
```

When to Use:

- ✅ When only part of method needs protection.
- ✅ To reduce lock contention.
- ✅ To lock on a specific object (not `this`).

Example:

```
public void add() {
  // Non-critical code (No lock)
  prepareData();
    synchronized(this) {
    // Critical section (Lock held)
    count++;
  }
    // Non-critical code (No lock)
  notifyUsers();
}
```

3.4 Static Synchronization

Theory: Static synchronization locks the Class object rather than an instance, ensuring that only one thread can execute any static synchronized method of that class across all instances. This is crucial for protecting static variables shared across all objects.

Definition: Synchronization that applies to the class level rather than instance level.

Internal Working:

- Lock Object: MyClass.class (The Class object representing the class in memory).
- Scope: Global across all instances of the class.
- Memory: Only one lock per class loader.

When to Use:

- ✅ Protecting static variables.
- ✅ Ensuring singleton thread safety.
- ✅ Global resource access (e.g., file writer).

Example:

```
public class Counter {
  private static int total = 0;
    // Locks Counter.class
  public static synchronized void increment() {
    total++;
  }
    // Equivalent block syntax
  public static void add() {
    synchronized(Counter.class) {
      total++;
    }
  }
}
```

4. Object Level Lock vs. Class Level Lock

Theory: Understanding the scope of the lock is vital for correct synchronization. Object-level locks protect instance-specific data, allowing concurrent access to different instances. Class-level locks protect static data, serializing access across all instances of the class.

4.1 Comparison Table

Feature	Object Level Lock	Class Level Lock
Lock On	Current Instance (this)	Class Object (MyClass.class)
Scope	Per Object	Global (All instances)
Methods	Instance synchronized methods	Static synchronized methods
Blocks	synchronized(this)	synchronized(MyClass.class)
Concurrency	Thread A locks Obj1, Thread B can lock Obj2	Thread A locks Class, Thread B waits
Use Case	Instance variables	Static variables

4.2 Internal Working Diagram

```
┌──────────────────────────────────────────────────┐
│              LOCK SCOPE COMPARISON                 │
├──────────────────────────────────────────────────┤
│                                                   │
│  Object Level Lock:                               │
│  ┌─────────┐       ┌─────────┐                    │
│  │  Obj A  │       │  Obj B  │                    │
│  │  [Lock] │       │  [Lock] │                    │
│  └─────────┘       └─────────┘                    │
│       │                 │                         │
│  Thread 1 (Locks A)  Thread 2 (Locks B) → ✅ Concurrent  │
│                                                   │
│  Class Level Lock:                                │
│  ┌──────────────────────────────┐                 │
│  │            MyClass.class     │                 │
│  │            [Global Lock]     │                 │
│  └──────────────────────────────┘                 │
│                    │                              │
│                    │                              │
│  Thread 1 (Locks Class)  Thread 2 (Waits) → ❌ Sequential  │
│                                                   │
└──────────────────────────────────────────────────┘
```

4.3 Code Example

```java
public class LockDemo {
    public synchronized void instanceMethod() {
        // Locks 'this'
    }

    public static synchronized void staticMethod() {
        // Locks 'LockDemo.class'
    }

    public void blockMethod() {
        synchronized(this) {
            // Locks 'this'
        }
        synchronized(LockDemo.class) {
            // Locks 'LockDemo.class'
        }
    }
}
```

5. Internal Working of Synchronization

5.1 Monitor Enter/Exit

Theory: The JVM implements synchronization using monitors. When a thread enters a synchronized block, it executes a monitorenter instruction. When it exits (normally or via exception), it executes monitorexit. The monitor ensures mutual exclusion and manages the wait/notify sets.

Definition: A runtime data structure that manages lock ownership and thread waiting queues.

Internal Steps:

1. **Check Lock:** Thread checks object header (Mark Word).
2. **Acquire:** If unlocked, thread attempts CAS to set lock flag.
3. **Block:** If locked, thread is added to **Entry Set** (Queue) and parked.
4. **Wait:** If wait() called, thread moves to **Wait Set** and releases lock.
5. **Notify:** notify() moves thread from Wait Set to Entry Set.
6. **Release:** On monitorexit, lock flag cleared, next thread in Entry Set unparked.

5.2 Memory Semantics (Happens-Before)

Theory: The Java Memory Model (JMM) defines a "happens-before" relationship that guarantees memory visibility. Synchronization actions (lock acquire/release) create happens-before edges, ensuring that writes before a release are visible to reads after a subsequent acquire.

Definition: A guarantee that memory writes by one statement are visible to another statement.

Rules:

1. **Monitor Lock Rule:** Unlock happens-before subsequent Lock on same monitor.
2. **Volatile Rule:** Write to volatile happens-before read of same volatile.
3. **Thread Start Rule:** start() happens-before any action in started thread.
4. **Thread Termination Rule:** All actions in thread happen-before join() returns.

Internal Working (Memory Barrier):

```
┌──────────────────────────────────────────────┐
│            MEMORY BARRIER (Flush)            │
├──────────────────────────────────────────────┤
│                                              │
│  Thread 1:                                   │
│  1. Update variable (in CPU Cache)           │
│  2. synchronized exit → MEMORY BARRIER       │
│  3. Flush Cache to Main Memory               │
│                                              │
│  Thread 2:                                   │
│  1. synchronized enter → MEMORY BARRIER      │
│  2. Invalidate CPU Cache                     │
│  3. Read from Main Memory (See updated value)│
│                                              │
└──────────────────────────────────────────────┘
```

Concurrency vs Parallelism

(Most misunderstood concept)

Concurrency — "Dealing with multiple things"

Definition:

Concurrency is about **structure** — handling multiple tasks that overlap in time. **Structure over time**: Organizing your program so multiple tasks can **make progress**, even if not at the exact same instant.

- It's about **managing access**, **coordination**, and **responsiveness**.
- Works **even on a single CPU core**.

Key idea

- Tasks **make progress together**
- Not necessarily at the same instant

How it works internally

- Single CPU core
- OS switches between threads using **time slicing**
- Fast context switching creates the illusion of simultaneity

Time →

T1: ▇▇▇▇ ▒▒▒▒ ▇▇▇▇ ▒▒▒▒

T2: ▒▒▒▒ ▇▇▇▇ ▒▒▒▒ ▇▇▇▇

📌 Only **one thread runs at a time**, but many are "alive"

Example

- UI thread + background thread
- Web server handling multiple connections on one core

Real-World Analogy

- You're cooking **and** talking on the phone.
  You chop veggies → pause → answer call → pause → stir soup → pause → say "bye"…
  You're **handling two tasks concurrently**, but **only doing one at a time**.

Parallelism — "Doing multiple things"

Definition:

Parallelism is about **execution** — doing multiple tasks **at the same time Actually running multiple tasks at the same physical time.**.

Key idea

- Requires **multiple core**
- Threads execute simultaneously

Core 1 → Thread A [redacted]

Core 2 → Thread B [redacted]

Example

- Image processing split across 8 cores
- Scientific computations

Real-World Analogy

You and your friend both cook in the same kitchen:

You chop onions (Core 1), they boil pasta (Core 2)—truly at the same time.

Concurrency vs Parallelism (Side-by-side)

Aspect	Concurrency	Parallelism
Goal	Manage tasks	Speed
Requires multi-core	❌	✅
Simultaneous execution	❌	✅
OS concept	Yes	Yes
Design complexity	High	Medium

📌 **You can have concurrency without parallelism**
📌 **You cannot have parallelism without concurrency**

Mental model (remember this)

Concurrency is **about time**
Parallelism is **about space (cores)**

The Relationship

✅ You can have concurrency WITHOUT parallelism

→ Single-core system handling 100 web requests via thread switching.

❌ You CANNOT have parallelism WITHOUT concurrency

→ To run tasks in parallel, you must first structure your program to handle multiple flows (i.e., make it concurrent).

🧠 **Think of it this way:**

Concurrency = Designing your app to handle multiple tasks.

Parallelism = Running those tasks simultaneously if hardware allows.

All parallel programs are concurrent, but not all concurrent programs are parallel.

Practical Implication for You (Java Developer)

If you're building a web service that handles many users:

→ Focus on concurrency (thread safety, async I/O, non-blocking design).

If you're processing large datasets or images:

→ Leverage parallelism (parallelStream, custom thread pools).

But beware:

Throwing more threads ≠ automatic speedup (due to contention, context switching, Amdahl's Law).

Parallelism adds complexity—only use when you measure a benefit.

Final Thought

Concurrency is about correctness under interleaving.

Parallelism is about performance through simultaneity.

You're asking exactly the right questions, Revathi. Mastering this distinction will help you design systems that are both safe and scalable.

Race Conditions

A race condition occurs when multiple threads read and write the same variable, i.e., they have access to shared data and try to change it at the same time. In such a scenario, threads are "racing" each other to access/change the data. This is a major security vulnerability. : A race condition occurs when the behavior of software depends on the relative timing of events, such as the order in which threads are scheduled to run. It happens when two or more threads access shared data concurrently, and at least one of them modifies the data, without proper synchronization.

Example: In the scenario below, Thread A and Thread B both access the same variable:

- Thread A reads a variable.
- Thread B reads the same variable before Thread A writes the new value.
- Thread A writes its value.
- Thread B writes its value, overwriting Thread A's update.

Result: The update from Thread A is lost.

Definition: A flaw where the system's substantive behavior is contingent on the timing or ordering of other uncontrollable events.

Why it Happens:

- **Non-Atomic Operations:** Operations like count++ look single but are actually three steps (Read, Modify, Write).
- **Context Switching:** OS can switch threads between these steps.

Why Race Conditions Occur

A race condition happens in Java when:

1. Multiple Threads Access Shared Mutable Data

If a variable, object, file, or database is shared among threads and at least one thread modifies it, there's a chance of conflict.

Example:

counter++; // Not atomic → Read, Increment, Write steps

2. Lack of Proper Synchronization

If shared data is accessed without synchronization mechanisms (synchronized, Lock, volatile, etc.), threads can interleave their operations unpredictably.

Example:

Thread A reads a variable, Thread B updates it before Thread A writes, leading to overwritten or inconsistent values.

3. Non-Atomic Operations

- Even seemingly simple operations like x++ or list.add() are not atomic in Java. They involve multiple low-level steps.
- Without synchronization, another thread can interrupt these steps.

Example of Race Condition

```
class Geeks{
  int count=0;
  public void increment() {
    count++;
  }}
```

```
public class RaceConditionExample {
  public static void main(String[] args) throws InterruptedException {
    Geeks counter = new Geeks();
    Thread t1 = new Thread(() -> {
      for (int i = 0; i < 1000; i++) {
        counter.increment();
      }
    });
    Thread t2 = new Thread(() -> {
      for (int i = 0; i < 1000; i++) {
        counter.increment();
      }
    });
    t1.start();
    t2.start();
    t1.join();
    t2.join();
    System.out.println("Final Count: " + counter.count);
  }
}
```

Output:

Final Count: 1527

Explanation:

- Geeks has a shared variable count and an increment() method that increases it by 1.
- The count++ operation is not atomic, so multiple threads can interfere with each other's updates.
- Two threads (t1 and t2) are created, each calling increment() 1000 times on the same Geeks object.
- Both threads are started, and join() is used so the main thread waits until they finish.
- The expected output is 2000, but due to race conditions the final count may be less than 2000.

Internal Working (The count++ Problem):

```
┌──────────────────────────────────────────────────────┐
│              RACE CONDITION INTERNAL                 │
├──────────────────────────────────────────────────────┤
│                                                      │
│  Operation: count++ (where count = 5)                │
│                                                      │
│  Step 1: READ   → Thread 1 reads 5                   │
│  Step 2: READ   → Thread 2 reads 5 (Context Switch!) │
│  Step 3: MODIFY → Thread 1 calculates 5 + 1 = 6      │
│  Step 4: MODIFY → Thread 2 calculates 5 + 1 = 6      │
│  Step 5: WRITE  → Thread 1 writes 6                  │
│  Step 6: WRITE  → Thread 2 writes 6 (Overwrites Thread 1!) │
│                                                      │
│  Result: count = 6 (Should be 7) → DATA CORRUPTION   │
│                                                      │
└──────────────────────────────────────────────────────┘
```

How to Fix: Use `synchronized`, `Lock`, or `AtomicInteger`.

Types of Race Conditions

Race conditions are generally classified into these main types:

1. Read-Modify-Write Race

Read-Modify-Write Race occurs when multiple threads read a variable, modify its value, and write it back without synchronization.

2. Check-Then-Act Race

Check-Then-Act Race occurs when a thread checks a condition and acts on it, but the condition changes before the action.

Example:

if (list.contains(item)) {

list.add(item); // Another thread could add it in between

}

3. Initialization Race

- It happens when multiple threads try to initialize a shared resource at the same time.
- **Example:** Lazy initialization of a singleton without proper synchronization.

4. Data Visibility Race

- One thread updates a variable, but another thread sees a stale value due to caching or reordering.
- Fixed using volatile or synchronization.

5. Order Violation Race

- When the correct functioning of the program depends on operations happening in a specific order, but threads run in a different order.

How to Prevent Race Conditions

1. Use Synchronization

Wrap critical sections in synchronized blocks or methods so only one thread can access them at a time.

Example:

```
public synchronized void increment() {
count++;
}
```

2. Use Locks (java.util.concurrent.locks.Lock)

Provides more flexibility than synchronized, with features like try-lock and timed locking.

Example:

```
Lock lock = new ReentrantLock();
lock.lock();
try {
count++;
} finally {
lock.unlock();
}
```

3. Use Atomic Variables

Classes like AtomicInteger, AtomicLong, etc., perform thread-safe atomic operations without explicit synchronization.

Example:

```
AtomicInteger count = new AtomicInteger();
count.incrementAndGet();
```

4. Use volatile for Visibility

- Ensures that changes to a variable are immediately visible to other threads, avoiding stale data issues.
- Note: volatile alone does not make compound actions atomic.

Example of Handle Race Condition

```
class Geeks{
  private int count = 0;
  // synchronized ensures only one thread executes this at a time
  public synchronized void increment() {
    count++;
  }
public synchronized int getCount() {
    return count;   }}
public class RaceConditionHandled {
  public static void main(String[] args) throws InterruptedException {
   Geeks counter = new Geeks();
    Thread t1 = new Thread(() -> {
      for (int i = 0; i < 1000; i++) {
        counter.increment();
      }
    });
    Thread t2 = new Thread(() -> {
      for (int i = 0; i < 1000; i++) {
        counter.increment();
      }
    });
    t1.start();     t2.start();     t1.join();     t2.join();
    System.out.println("Final Count (Handled): " + counter.getCount());
  }}
```

Output

Final Count (Handled): 2000

2.2 Critical Section

Theory: A critical section is a segment of code that accesses shared resources (variables, files, network connections) and must not be executed by more than one thread at a time. Identifying and protecting critical sections is the fundamental step in designing thread-safe programs.

Definition: A code segment where shared resources are accessed and which must be executed atomically.

Why We Use:

- To isolate unsafe code.
- To minimize lock scope (performance optimization).

How to Identify:

1. Look for shared variables (static fields, instance fields accessed by multiple threads).
2. Look for compound operations (check-then-act, read-modify-write)

```
// ❌ Critical Section NOT Protected
public void unsafe() {
  if (balance >= amount) {     // Step 1: Check
    balance -= amount;       // Step 2: Act (Race condition here!)
  }
}
// ✅ Critical Section Protected
public synchronized void safe() {
  if (balance >= amount) {     // Entire block is atomic
    balance -= amount;
  }
}
```

2.3 Atomic Operations

Theory: An atomic operation is an operation that completes in a single step relative to other threads. Either the operation happens completely, or it doesn't happen at all; there is no visible intermediate state. Atomicity is essential for maintaining data integrity in concurrent environments.

Definition: An operation that is indivisible and uninterruptible.

Definition: An operation that executes as a single, uninterruptible unit relative to other threads. Either the entire operation succeeds, or it fails without side effects; there is no visible intermediate state.

Why We Use?

- **Performance:** No thread blocking or context switching (lock-free).
- **Deadlock Freedom:** No locks means no possibility of deadlocks.
- **Scalability:** Better throughput under high contention compared to synchronized.
- **Fine-Grained:** Locks only the specific variable, not the entire object.

When to Use?

- ✅ **Counters:** Incrementing/decrementing shared counters.
- ✅ **Flags:** Setting status flags safely.
- ✅ **Sequence Generators:** Generating unique IDs.
- ❌ **Compound Actions:** If you need to update multiple variables atomically (use synchronized or Lock).
- ❌ **Complex Logic:** If the update logic depends on multiple steps (use synchronized).

Types:

- **Primitive Assignment:** x = 5 (atomic for int, not for long/double on 32-bit JVM).
- **Reference Assignment:** ref = new Object() (atomic).
- **Volatile Read/Write:** (atomic).
- **Atomic Classes:** AtomicInteger.incrementAndGet() (atomic via CAS).

Why count++ is NOT Atomic:

- It involves **Read** (get value), **Modify** (add 1), **Write** (store value).
- Threads can interleave between these steps.

- **Performance:** Faster than `synchronized` for simple counters/flags.
- **No Deadlock:** Lock-free algorithms cannot deadlock.
- **Fine-Grained:** Locks only the specific variable, not the whole object.
-

Internal Working (CAS):

Theory: CAS is the fundamental CPU instruction that enables atomic operations in Java. It is a hardware-level instruction that compares the value in a memory location with a given expected value and, only if they match, modifies the memory location to a new given value. This operation is performed atomically by the processor.

Definition: A concurrency control method used to achieve synchronization in a multi-threaded environment without using locks.

```
┌──────────────────────────────────────────────────┐
│              ATOMIC OPERATION (CAS)              │
├──────────────────────────────────────────────────┤
│                                                  │
│  Compare-And-Swap (CPU Instruction):             │
│                                                  │
│  1. Read current value from memory (5)           │
│  2. Calculate new value (6)                      │
│  3. CAS Instruction:                             │
│     IF memory_value == expected_value (5)        │
│     THEN write new_value (6)                     │
│     ELSE retry                                   │
│                                                  │
│  Benefit: No locking overhead, hardware supported.  │
│                                                  │
```

Internal working

```
// AtomicInteger.incrementAndGet() logic

public final int incrementAndGet() {

  for (;;) {

    int current = get();

    int next = current + 1;

    if (compareAndSet(current, next)) // CAS

      return next;

    // If CAS fails (another thread updated), retry loop

  }

}

public void increment() {
  while (true) {
    int current = counter.get();
    int next = current + 1;
    if (counter.compareAndSet(current, next)) {
      return;
```

```
        }
        Thread.onSpinWait();
        // else: someone updated it → retry with fresh value
    }
}
```

When to Use:

- ✅ Counters, flags, sequence generators.
- ✅ High contention, low complexity updates.
- ❌ Complex compound actions (use synchronized or Lock).

nternal Working (CPU Level):

```
┌──────────────────────────────────────────────┐
│              CAS INTERNAL WORKING (CPU)      │
├──────────────────────────────────────────────┤
│                                              │
│  Instruction: CMPXCHG (Compare and Exchange) │
│                                              │
│  Inputs:                                     │
│  1. Memory Address (V)                       │
│  2. Expected Value (A)                       │
│  3. New Value (B)                            │
│                                              │
│  Logic:                                      │
│  IF (Value at V == A) THEN                   │
│      Write B to V                            │
│      Return SUCCESS (true)                   │
│  ELSE                                        │
│      Do NOT write                            │
│      Return FAILURE (false)                  │
│      (Update A to current value at V)        │
│                                              │
│  Atomicity: Guaranteed by CPU hardware bus locking. │
│                                              │
└──────────────────────────────────────────────┘
```

ava Implementation (`Unsafe` class):

```java
// Simplified internal logic of AtomicInteger
public final int getAndIncrement() {
    return unsafe.getAndAddInt(this, valueOffset, 1);
}

// Inside Unsafe (Native Method)
public final int getAndAddInt(Object var1, long var2, int var4) {
    int var5;
    do {
        var5 = this.getIntVolatile(var1, var2); // Read current
    } while (!this.compareAndSwapInt(var1, var2, var5, var5 + var4)); // CAS
    return var5;
}
```

- **Loop:** If CAS fails (another thread modified value), retry loop (Spinlock).
- **Offset:** `valueOffset` is the memory address of the `value` field calculated via reflection.

3. Atomic Classes Hierarchy

Theory: Java provides a suite of atomic classes in java.util.concurrent.atomic that wrap primitive types and object references. These classes expose methods like incrementAndGet, compareAndSet, and getAndUpdate which internally utilize the Unsafe class to perform CAS operations.

3.1 Basic Types

Class	Type	Key Methods	Use Case
AtomicBoolean	boolean	compareAndSet, getAndSet	Flags, states
AtomicInteger	int	incrementAndGet, addAndGet	Counters, IDs
AtomicLong	long	incrementAndGet, addAndGet	High-range counters
AtomicReference<V>	Object	compareAndSet, getAndUpdate	Safe object publication

- **Volatile:** The underlying value is volatile to ensure visibility across threads.
- **Unsafe:** Uses sun.misc.Unsafe (or jdk.internal.misc.Unsafe) to access memory directly.

3.2 Array Types

Class	Type	Key Methods	Use Case
AtomicIntegerArray	int[]	getAndSet(index, val)	Atomic array updates
AtomicLongArray	long[]	compareAndSet(index, expect, update)	Atomic long array
AtomicReferenceArray	Object[]	getAndUpdate(index, function)	Atomic object array

Why Use?

- Standard arrays are not thread-safe.
- synchronized locks the whole array.
- Atomic arrays lock only the specific index (via CAS).

3.3 Field Updaters

Class	Purpose	Key Methods
AtomicIntegerFieldUpdater	Atomic update on existing volatile int field	compareAndSet(obj, expect, update)
AtomicLongFieldUpdater	Atomic update on existing volatile long field	compareAndSet(obj, expect, update)
AtomicReferenceFieldUpdater	Atomic update on existing volatile reference field	compareAndSet(obj, expect, update)

Why Use?

- Avoids creating wrapper objects (AtomicInteger) for every instance.
- Useful when you cannot modify the original class to extend Atomic classes.
- **Requirement:** The field must be volatile.

3.4 Advanced Atomics (ABA Problem)

Theory: The ABA problem occurs in CAS operations when a value changes from A to B and back to A before the CAS operation completes. The CAS operation sees "A" and succeeds, assuming nothing changed, but intermediate state changes might have invalidated the operation context (e.g., a node was removed and re-added in a linked list).

Definition: A scenario where a memory location is read as value A, changed to B, then changed back to A, fooling a CAS check into thinking no change occurred.

Solution Classes:

Class	Mechanism	Use Case
`AtomicStampedReference<V>`	Adds an **integer stamp** (version) to the reference	Solves ABA for references
`AtomicMarkableReference<V>`	Adds a **boolean mark** to the reference	Track if node was deleted/modified

4. Atomic Methods Reference

Method	Description	Internal Logic	Returns
get()	Gets current value	Volatile read	int/T
set(int newValue)	Sets value	Volatile write	void
compareAndSet(expect, update)	CAS operation	Unsafe CAS	boolean
getAndSet(int newValue)	Sets & returns old	CAS loop	int (old)
getAndIncrement()	Increments & returns old	getAndAdd(1)	int (old)
incrementAndGet()	Increments & returns new	getAndAdd(1) + 1	int (new)
getAndUpdate(Function)	Updates via function	CAS loop with lambda	int (old)
updateAndGet(Function)	Updates via function	CAS loop with lambda	int (new)
accumulateAndGet(int, Function)	Accumulates value	CAS loop with binary op	int (new)

2.4 Visibility Issues

Theory: Visibility problems arise when changes made to a variable by one thread are not immediately visible to other threads due to CPU caching architectures. Modern CPUs use local caches to speed up memory access, which can lead to threads working with stale data unless synchronization mechanisms force updates to main memory.

Definition: The guarantee that a write to a variable by one thread is seen by other threads.

Why it Happens:

- **CPU Caches:** Each thread may cache variables in registers or L1/L2 cache.
- **Compiler Optimizations:** Instructions may be reordered.
- **No Flush:** Without sync, writes may stay in local cache.

Internal Working (Main Memory vs. Working Memory):

```
┌──────────────────────────────────────────────┐
│              VISIBILITY PROBLEM              │
├──────────────────────────────────────────────┤
│                                              │
│  Thread 1                    Main Memory     │
│  ┌─────────┐                 ┌──────────┐    │
│  │ Cache   │                 │ Variable │    │
│  │ (flag=T) │ ──────────────→│ (flag=F) │    │
│  └─────────┘                 └──────────┘    │
│       ↑                          ↑           │
│       |                          |           │
│  Thread 2                    (Stale Value)   │
│  ┌─────────┐                                 │
│  │ Cache   │                                 │
│  │ (flag=F) │ ← Reads old value from its own cache │
│  └─────────┘                                 │
│                                              │
│  Solution: synchronized or volatile forces flush to Main │
│  Memory, invalidating other caches.          │
│                                              │
└──────────────────────────────────────────────┘
```

How to Fix:

- `synchronized`: Flushes cache on lock acquire/release.
- `volatile`: Forces read/write directly to main memory.

```
// ❌ Visibility Problem
private boolean running = true; // Not volatile
public void stop() { running = false; } // May stay in cache
public void run() {
  while (running) { // May never see false
    // Work
  }
}
```

7.3 Volatile Internal Working

```
┌──────────────────────────────────────────────┐
│              VOLATILE INTERNAL               │
├──────────────────────────────────────────────┤
│                                              │
│  Read:                                       │
│  1. Force read from Main Memory (not cache)  │
│  2. Insert Load Barrier                      │
│                                              │
│  Write:                                      │
│  1. Force write to Main Memory               │
│  2. Insert Store Barrier                     │
│  3. Invalidate other CPU caches              │
│                                              │
│  Result: Immediate visibility across threads.│
│                                              │
└──────────────────────────────────────────────┘
```

VOLATILE KEYWORD

1. Introduction to Volatile

Theory: The volatile keyword in Java provides a lightweight synchronization mechanism that guarantees visibility and ordering but not atomicity. It instructs the JVM and the CPU that a variable's value may be modified by different threads, preventing caching optimizations that could lead to stale data reads.

Definition: A modifier applied to fields that ensures changes to the variable are immediately visible to all threads and prevents instruction reordering around the variable access.

Why We Use?

- **Visibility:** Ensure updates are seen by other threads immediately.
- **Ordering:** Prevent compiler/CPU from reordering instructions (Memory Barriers).
- **Lightweight:** Cheaper than synchronized (no locking overhead).
- **Safe Publication:** Safely publish objects without full synchronization.

When to Use?

- ✅ **Status Flags:** volatile boolean running.
- ✅ **One-time Publication:** Singleton DCL (Double-Checked Locking).
- ✅ **Independent Variables:** When variable update does not depend on current value.
- ❌ **Compound Actions:** count++ (Not atomic).
- ❌ **Dependent Updates:** if (x > y) x = y (Requires lock).

---

2. Internal Working of Volatile

Theory: Volatile works by inserting **Memory Barriers** (also called Memory Fences) around read and write operations. These barriers prevent the compiler and CPU from reordering instructions and force data to be flushed to or read from main memory rather than CPU caches.

Definition: CPU instructions that enforce a specific ordering of memory operations.

Internal Working (Memory Barriers):

```
┌──────────────────────────────────────────────┐
│              VOLATILE MEMORY BARRIERS        │
├──────────────────────────────────────────────┤
│                                              │
│  Write Barrier (Store Barrier):              │
│  1. Write value to CPU Cache                 │
│  2. FLUSH cache to Main Memory               │
│  3. Invalidate other CPU caches              │
│                                              │
│  Read Barrier (Load Barrier):                │
│  1. Invalidate CPU Cache                     │
│  2. Read value from Main Memory              │
│                                              │
│  Reordering Prevention (Happens-Before):     │
│  - No write can be reordered after a volatile write  │
│  - No read can be reordered before a volatile read   │
│                                              │
│  Assembly (x86):                             │
│  volatile write → LOCK prefix instruction (e.g., LOCK ADD)  │
│  volatile read  → Usually just a normal read (due to cache) │
│                                              │
└──────────────────────────────────────────────┘
```

Java Memory Model (JMM) Guarantees:

1. **Visibility:** Write to volatile happens-before read of same volatile.
2. **Ordering:** Instructions before volatile write cannot move after it. Instructions after volatile read cannot move before it.
3. **Atomicity: NOT Guaranteed** for compound actions (only single read/write is atomic).

---

3. Volatile vs Synchronized

Feature	volatile	synchronized
Visibility	✅ Yes	✅ Yes
Atomicity	❌ No (except read/write)	✅ Yes (for critical section)
Ordering	✅ Yes (Memory Barriers)	✅ Yes (Happens-Before)
Blocking	❌ No (Non-blocking)	✅ Yes (Blocks thread)
Performance	Faster (No context switch)	Slower (Lock acquisition)
Scope	Variable level	Method/Block level
Null Handling	Can be null	Can be null
Use Case	Flags, single variables	Complex logic, compound actions

5. Volatile Limitations (The Atomicity Trap)

Theory: A common misconception is that volatile makes operations atomic. It does not. Operations like count++ involve three steps (Read, Modify, Write). Volatile ensures each step sees the latest value, but it does not prevent another thread from intervening between the steps.

Definition: Volatile guarantees visibility of individual reads/writes, but not atomicity of compound actions.

Internal Working (Why count++ Fails):

```
┌──────────────────────────────────────────────┐
│              VOLATILE ATOMICITY FAILURE      │
├──────────────────────────────────────────────┤
│                                              │
│  Operation: volatileCount++                  │
│                                              │
│  Thread 1:                                   │
│  1. Read volatileCount (5) → Main Memory     │
│  2. Calculate 5 + 1 = 6                      │
│  3. [CONTEXT SWITCH]                         │
│  4. Write 6 to volatileCount → Main Memory   │
│                                              │
│  Thread 2 (Intervenes):                      │
│  1. Read volatileCount (5) → Main Memory     │
│  2. Calculate 5 + 1 = 6                      │
│  3. Write 6 to volatileCount → Main Memory   │
│                                              │
│  Result: 6 (Should be 7). Volatile ensured visibility,  │
│  but did not prevent interleaving.           │
│                                              │
└──────────────────────────────────────────────┘
```

Solution: Use `AtomicInteger` or `synchronized`.

6. Happens-Before Relationship

Theory: The Java Memory Model defines a "happens-before" relationship that guarantees memory visibility. If Action A happens-before Action B, then the results of A are visible to B. Volatile writes and reads establish specific happens-before edges.

Definition: A partial ordering of actions in the JMM that guarantees memory visibility.

Why It Matters:

- **Visibility:** Ensures writes are seen by other threads.
- **Ordering:** Prevents harmful reordering.
- **Correctness:** Foundation for writing correct concurrent code

Volatile Rules:

1. **Write-Read:** A write to a volatile field happens-before every subsequent read of that same field.
2. **Transitivity:** If A happens-before B, and B happens-before C, then A happens-before C.

Internal Diagram

```
┌──────────────────────────────────────────────────┐
│              HAPPENS-BEFORE (Volatile)           │
├──────────────────────────────────────────────────┤
│                                                  │
│  Thread 1                    Thread 2            │
│  ┌─────────┐                 ┌─────────┐         │
│  │ x = 1   │                 │         │         │
│  │ v = true │ ──HB Edge ──→ │ r = v   │         │
│  │         │   (Volatile)    │ if (r)  │         │
│  │         │                 │   y = x │         │
│  └─────────┘                 └─────────┘         │
│                                                  │
│  Guarantee:                                      │
│  Because v=true HB r=v, the write x=1 is visible to y=x.
│  Without volatile, y could see 0 (default).      │
│                                                  │
└──────────────────────────────────────────────────┘
```

1. Decision Matrix: Which to Use?

Scenario	Recommended	Why
Simple Counter	AtomicInteger / LongAdder	Lock-free, atomic, high performance
High Contention Counter	LongAdder	Striped cells reduce contention
Status Flag	volatile boolean	Visibility only, cheapest option
Singleton Instance	volatile + DCL	Safe publication, lazy init
Compound Action	synchronized / Lock	Atomicity required for multiple steps
Multiple Variables	synchronized / Lock	Need atomic update across variables
Object Reference	AtomicReference	CAS on objects, safe update
ABA Problem	AtomicStampedReference	Versioning prevents stale CAS

1. Atomic Classes Summary

Class	Type	Atomicity	Visibility	Ordering	Lock-Free
int (primitive)	Primitive	❌ (except read/write)	❌	❌	N/A
volatile int	Primitive	❌ (except read/write)	✅	✅	✅
AtomicInteger	Class	✅ (CAS)	✅	✅	✅
synchronized	Block	✅ (Lock)	✅	✅	❌

2. Volatile Guarantees

Guarantee	Description	Mechanism
Visibility	Write seen by all threads	Flush to Main Memory
Ordering	No reordering around access	Memory Barriers
Atomicity	Single read/write only	CPU Instruction
Happens-Before	Establishes ordering	JMM Rules

3. Internal Instructions (x86)

Java Operation	Assembly Instruction	Effect
volatile write	LOCK ADD [mem], 0	Locks bus, flushes cache
volatile read	MOV [reg], [mem]	Reads from cache (usually)
Atomic CAS	LOCK CMPXCHG	Compare & Swap atomically
Thread.yield	PAUSE	Hint to processor
LockSupport.park	syscall	OS Thread suspend

JAVA MEMORY MODEl

The **Java Memory Model (JMM)** is the foundation of all concurrency in Java. It defines how threads interact through memory, what guarantees Java provides for visibility and ordering, and how to write correct concurrent programs. This module covers the complete theoretical and practical understanding of JMM.

PART 1: INTRODUCTION TO JMM

1. What is Java Memory Model?

Theory: The Java Memory Model is a formal specification that defines the behavior of shared memory in multithreaded Java programs. It establishes rules for how threads interact through memory, what operations are atomic, and when changes made by one thread become visible to other threads. The JMM abstracts away hardware differences to provide consistent concurrency semantics across all platforms.

Definition: A specification that defines the relationship between variables stored in main memory and how threads read/write those variables through their working memory (CPU caches).

When to Understand JMM?

- ✅ Writing concurrent/multithreaded applications
- ✅ Debugging race conditions and visibility bugs
- ✅ Implementing lock-free algorithms
- ✅ Understanding volatile, synchronized, final semantics
- ✅ Performance tuning concurrent code

Why We Need JMM?

```
┌──────────────────────────────────────────────────────────┐
│                WHY JAVA MEMORY MODEL?                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Problem: Hardware Memory Architecture                   │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐               │
│  │  CPU 1   │  │  CPU 2   │  │  CPU 3   │               │
│  │ Cache L1 │  │ Cache L1 │  │ Cache L1 │               │
│  │ Cache L2 │  │ Cache L2 │  │ Cache L2 │               │
│  └──────────┘  └──────────┘  └──────────┘               │
│       └──────────────┴──────────────┘                    │
│                       ↓                                  │
│                 ┌──────────┐                             │
│                 │ Main Memory │                           │
│                 │  (RAM)   │                             │
│                 └──────────┘                             │
│                                                          │
│  Issues Without JMM:                                     │
│  1. Different CPUs have different memory models          │
│  2. Compiler optimizations may reorder instructions      │
│  3. CPU caches may not flush to main memory promptly     │
│  4. Same Java code behaves differently on different HW   │
│                                                          │
│  Solution: JMM provides consistent guarantees across all │
│  platforms regardless of underlying hardware.            │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2. Key Concepts of JMM

2.1 Main Memory vs. Working Memory

Theory: JMM abstracts memory into two types: Main Memory (shared by all threads) and Working Memory (private to each thread). All variables are stored in Main Memory, but threads operate on copies in their Working Memory. Synchronization actions control when data is flushed between these memories.

Definition:

- **Main Memory:** Shared memory where all variables are stored (heap).
- **Working Memory:** Thread-local memory (CPU caches, registers) where threads operate on variables.

Internal Working:

```
┌──────────────────────────────────────────────────────────┐
│                JMM MEMORY ARCHITECTURE                   │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌──────────────────────────────────────────────┐        │
│  │                  Main Memory                 │        │
│  │             (Shared Heap - All Variables)    │        │
│  └──────────────────────────────────────────────┘        │
│          ↑                  ↑                  ↑         │
│          │ Load/Store       │ Load/Store       │         │
│          ↓                  ↓                  ↓         │
│  ┌──────────┐   ┌──────────┐   ┌──────────┐             │
│  │ Thread 1 │   │ Thread 2 │   │ Thread 3 │             │
│  │ Working Mem │ │ Working Mem │ │ Working Mem │         │
│  │ (CPU Cache) │ │ (CPU Cache) │ │ (CPU Cache) │         │
│  └──────────┘   └──────────┘   └──────────┘             │
│                                                          │
│ Operations:                                              │
│ - load: Main Memory → Working Memory                     │
│ - store: Working Memory → Main Memory                    │
│ - read: Working Memory → Thread                          │
│ - write: Thread → Working Memory                         │
│ - lock/unlock: Synchronization actions                   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

JMM Rules:

1. All variables reside in Main Memory.
2. Each thread has its own Working Memory.
3. Threads cannot directly access other threads' Working Memory.
4. All variable access must go through Working Memory.
5. Synchronization controls flushing between memories.

Part 1: Inter-Thread Communication (ITC)

In a multi-threaded environment, threads often need to coordinate. One thread might need to wait for another thread to complete a task or change a state before it can proceed. This coordination is called Inter-Thread Communication.

Why not Polling? A naive approach is "Polling" (checking a flag in a while loop).

- **Problem:** It wastes CPU cycles (Busy Waiting).
- **Solution:** ITC allows a thread to **sleep** efficiently until notified, freeing up the CPU.

1.2 The Mechanism: Object Monitors

In Java, ITC is built on the **Monitor Pattern**.

- Every Java Object has an associated **Monitor** (or Intrinsic Lock).
- A thread must own the monitor to enter a synchronized block/method on that object.
- The Monitor manages three sets of threads:
    1. **Entry Set:** Threads waiting to acquire the lock.
    2. **Wait Set:** Threads that have released the lock and are waiting for a notification.

3. **Owner:** The thread currently holding the lock.

1. The Core Mechanisms: wait(), notify(), notifyAll()

These methods are defined in the java.lang.Object class, **not** the Thread class. This is because they operate on the **monitor lock** of a specific object.

Definitions

Method	Definition	Behavior
wait()	Causes the current thread to wait until another thread invokes notify() or notifyAll() on this object.	1. Releases the lock on the object. 2. Pauses execution. 3. Must be called inside a synchronized block/method.
notify()	Wakes up a **single** thread that is waiting on this object's monitor.	1. Does not release the lock immediately (releases after the synchronized block exits). 2. Which thread wakes up is arbitrary.
notifyAll()	Wakes up **all** threads that are waiting on this object's monitor.	1. All threads compete for the lock again. 2. Safer than notify() in most scenarios.

Critical Theory: The Monitor & Spurious Wakeups

1. **Monitor Lock:** To call wait(), a thread must own the monitor lock of the object. If it doesn't, an IllegalMonitorStateException is thrown.
2. **Spurious Wakeups:** A thread can wake up from wait() without being notified. Therefore, wait() should **always** be called inside a while loop checking the condition, never an if statement.
    - *Wrong:* if (condition) wait();
    - *Right:* while (condition) wait();

Where and How to Use

- **Where:** Inside synchronized methods or blocks.
- **How:** Use when a thread needs to pause because a resource is unavailable (e.g., buffer full, buffer empty, data not ready).

1.4 Critical Best Practices

1. The "Wait Loop" Idiom

Never use if to check the condition before wait(). Always use while.

Reason: Spurious Wakeups. The JVM/OS may wake a thread without notify() being called.

Reason: Stale State. By the time the thread wakes up and re-acquires the lock, another thread might have changed the state again.

```
// WRONG
if (buffer.isEmpty()) {
  wait();
}
// CORRECT
while (buffer.isEmpty()) {
  wait();
}
```

2. Synchronization Requirement

Calling wait() without holding the object's lock throws IllegalMonitorStateException

2. The Producer-Consumer Problem

This is the classic ITC problem.

- **Producer:** Generates data and puts it into a shared buffer.
- **Consumer:** Takes data from the buffer and processes it.
- **Constraint:** Producer must wait if buffer is full. Consumer must wait if buffer is empty.

Implementation using wait/notify

```
import java.util.LinkedList;
import java.util.Queue;
public class ProducerConsumer {
  private final Queue<Integer> queue = new LinkedList<>();
  private final int CAPACITY = 5;
  public void produce() throws InterruptedException {
    int value = 0;
    while (true) {
      synchronized (this) {
        // WAIT if buffer is full
```

```
            while (queue.size() == CAPACITY) {
                wait();
            }
            queue.add(value++);
            // Notify consumer that data is available
            notifyAll();
        }
        // Simulate work
        Thread.sleep(100);
    }
  }
  public void consume() throws InterruptedException {
    while (true) {
        synchronized (this) {
            // WAIT if buffer is empty
            while (queue.isEmpty()) {
                wait();
            }
            int val = queue.poll();
            // Notify producer that space is available
            notifyAll();
        }
        // Simulate processing
        Thread.sleep(100);
    }
  }
}
```

2.3 Internal Flow

1. **Producer** acquires lock, adds item, calls notifyAll(), releases lock.
2. **Consumer** (waiting in Wait Set) wakes up, tries to acquire lock.

3. **Consumer** acquires lock, checks while condition (is it empty?), consumes item.

Why notifyAll() instead of notify()?

If you use notify(), you might wake up a thread that cannot proceed (e.g., waking a Producer when the buffer is still full because a Consumer hasn't run yet). This can lead to Lost Wakeups, where threads wait forever. notifyAll() ensures all waiting threads check the condition again.

Modern Alternatives (Preferred in Production) **2.4 Modern Alternative: BlockingQueue**

Using wait/notify is low-level and error-prone. Java 5+ introduced the java.util.concurrent package.

Alternative	Usage	Benefit
BlockingQueue	Producer-Consumer	Handles locking/waiting internally. (e.g., ArrayBlockingQueue).
Semaphore	Limit access to resources	Permits based access.
Condition	Replacement for wait/notify	Works with ReentrantLock. Allows multiple wait-sets (more granular than Object monitor).
CountDownLatch	Wait for tasks to finish	One-time event signal.

Example using BlockingQueue:

```
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);

// Producer

queue.put(value); // Blocks automatically if full

// Consumer

int val = queue.take(); // Blocks automatically if empty
```

4.1 Internal Working of wait()

When wait() is called:

1. **Check Ownership:** JVM checks if current thread owns the monitor. If not -> IllegalMonitorStateException.
2. **Add to Wait Set:** The thread is added to the Object's Wait Set.
3. **Release Lock:** The monitor's lock count is decremented.

4. **Park:** The OS parks (suspends) the thread. It consumes 0% CPU.
5. **Notification:** When notify() is called, the thread is moved from Wait Set to **Entry Set**.
6. **Re-contention:** The thread must compete with other threads to re-acquire the lock.
7. **Resume:** Once lock is acquired, wait() returns, and execution continues after the wait() call.

4.2 Why notify() is dangerous (The "Lost Wakeup" Problem)

Scenario:

1. Thread A and Thread B are waiting on Object X.
2. Thread C calls notify(). Thread A wakes up.
3. Thread A checks condition, finds it false (maybe Thread D changed state), and goes back to wait().
4. Thread B is still sleeping.
5. Thread C finishes. No one else calls notify().
6. **Result:** Thread B sleeps forever.

Solution: Always use notifyAll() unless you are 100% sure only one thread type is waiting, or use Condition variables (see below).

Part 2: Thread Problems

When threads interact improperly, specific pathological states occur.

1. Deadlock

Definition : A situation where two or more threads are blocked forever, waiting for each other to release locks.

.

The 4 Coffman Conditions (Necessary for Deadlock)

For a deadlock to occur, **all four** of these conditions must hold simultaneously:

1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode (only one thread at a time).
2. **Hold and Wait:** A thread holding at least one resource is waiting to acquire additional resources held by other threads.
3. **No Preemption:** Resources cannot be forcibly taken from a thread; they must be released voluntarily.
4. **Circular Wait:** A set of threads exists such that T1 waits for T2, T2 waits for T3, ..., Tn waits for T1.

Example Scenario

- Thread A holds Lock 1, wants Lock 2.
- Thread B holds Lock 2, wants Lock 1.
- Both wait forever.

Detection Techniques

1. **Thread Dump:** In Java, use jstack <pid> or JConsole/VisualVM. Look for state BLOCKED and "Found one Java-level deadlock".
2. **Resource Allocation Graph:** An algorithmic approach (nodes = threads/resources, edges = requests/assignments). A cycle indicates deadlock.
3. **Monitoring Tools:** APM tools (New Relic, Dynatrace) can detect long-held locks.

Prevention Strategies (Breaking the Coffman Conditions)

1. **Lock Ordering (Breaks Circular Wait):** Always acquire locks in a predefined order.
    - *Rule:* If you need Lock A and Lock B, always lock A first, then B. Never B then A.
2. **TryLock with Timeout (Breaks Hold and Wait):** Use ReentrantLock.tryLock(timeout). If the lock isn't available within the time, release held locks and retry later.
3. **Avoid Nested Locks:** Don't acquire a second lock while holding the first.
4. **Use High-Level Concurrency:** Use ConcurrentHashMap, BlockingQueue, etc., which manage locking internally.

---

2. Starvation

Definition

Starvation occurs when a thread is unable to gain regular access to shared resources and is unable to make progress. This typically happens when resources are denied due to aggressive behavior by other threads.

Causes

1. **Thread Priority:** Low-priority threads are constantly preempted by high-priority threads.
2. **Indefinite Waiting:** A thread waits on a resource that is never notified (e.g., using wait() without a corresponding notify()).
3. **Unfair Locking:** If a locking mechanism favors specific threads (e.g., LIFO instead of FIFO), some threads may never get the lock.

Prevention Strategies

1. **Avoid Priority Manipulation:** Do not rely on Thread.setPriority(). Let the OS scheduler decide.

2. **Use Fair Locks:** In ReentrantLock, use the fairness policy: new ReentrantLock(true). This grants the lock to the longest-waiting thread (FIFO).
3. **Minimize Synchronized Blocks:** Keep critical sections short so threads hold locks for less time.

---

3. Livelock

Definition

Livelock is similar to deadlock, but the threads are **not blocked**. They are actively running and changing state, but they are making **no progress**.

Analogy

Two people meet in a narrow hallway.

- Person A steps left to let B pass.
- Person B steps left to let A pass.
- They collide again.
- Person A steps right.
- Person B steps right.
- They collide again.
- They keep moving (active) but never pass (no progress).

In Code

Usually occurs in retry logic.

```
// Pseudo-code for Livelock
while (resourceBusy) {
  releaseMyResource();
  waitRandomTime();
  tryAcquireResource();
}
// If two threads do this in sync, they keep releasing and retrying forever.
```

Difference from Deadlock

- **Deadlock:** Threads are in BLOCKED or WAITING state. CPU usage is low.
- **Livelock:** Threads are in RUNNABLE state. CPU usage is high (busy looping).

Prevention Strategies

1. **Random Backoff:** Introduce randomness in the retry delay so threads don't sync up their retries.
2. **Coordinator Election:** Elect one thread to act as the leader to decide who proceeds.
3. **Retry Limits:** Stop retrying after N attempts and throw an exception or escalate.

Part 3: Summary & Comparison

Comparison Table

Feature	Deadlock	Starvation	Livelock
State	Blocked / Waiting	Waiting / Ready	Runnable (Active)
Progress	None	None (for specific thread)	None (system-wide)
CPU Usage	Low	Low/Medium	High (Busy Waiting)
Cause	Circular dependency on locks	Priority / Unfair scheduling	Aggressive retry logic
Fix	Lock Ordering, Timeouts	Fair Locks, Avoid Priorities	Random Backoff

Best Practices Checklist for Developers

1. **Prefer java.util.concurrent:** Avoid raw wait/notify and synchronized where possible. Use ExecutorService, BlockingQueue, CompletableFuture.
2. **Lock Ordering:** If you must use multiple locks, define a global order and stick to it.
3. **Timeouts:** Always use timeouts when waiting for locks (tryLock) or I/O.
4. **Immutability:** Immutable objects are inherently thread-safe and require no locking.
5. **Thread Dumps:** Know how to generate a thread dump (kill -3 <pid> or jstack) for production debugging.
6. **Avoid notify():** Default to notifyAll() unless you are 100% sure only one specific waiter can proceed, to avoid lost wakeups.

When to use what?

Scenario	Recommended Tool
Simple Shared Counter	AtomicInteger
Producer-Consumer	BlockingQueue (e.g., LinkedBlockingQueue)
Wait for N tasks to finish	CountDownLatch or CyclicBarrier

Scenario	Recommended Tool
Limit concurrent access	Semaphore
Complex Condition Waiting	ReentrantLock with Condition
Read-Heavy Data	ReadWriteLock or StampedLock

Raw wait/notify is considered low-level and difficult to get right. Java 5 introduced the java.util.concurrent (JUC) package.

1. ReentrantLock & Condition

More flexible than synchronized. Allows multiple wait sets per lock.

Java

```
Lock lock = new ReentrantLock();

Condition notFull = lock.newCondition();

Condition notEmpty = lock.newCondition();


// Producer

lock.lock();

try {

   while (count == MAX) notFull.await(); // Similar to wait()

   // add item

   notEmpty.signal(); // Similar to notify()

} finally {

   lock.unlock();

}
```

- **Benefit:** You can have specific conditions (notFull vs notEmpty). signal() on notFull only wakes producers, not consumers. This is more efficient than notifyAll().

2. Semaphore

Controls access to a pool of resources (not just 1).

- **Use Case:** Limiting database connections to 10.
- **Method:** acquire() (decrements permit, blocks if 0), release() (increments permit).
-

3. CountDownLatch / CyclicBarrier

- **CountDownLatch:** One or more threads wait until a set of operations performed in other threads completes.
- **CyclicBarrier:** A set of threads wait for each other to reach a common barrier point.

4. volatile Keyword

- **Not for ITC:** volatile guarantees **Visibility** (changes seen by all threads) and prevents **Instruction Reordering**.
- **Does NOT guarantee Atomicity:** i++ is not safe with just volatile.
- **Use Case:** Status flags (e.g., volatile boolean running = true;).

4.4 Summary Comparison Table

Feature	synchronized + wait/notify	ReentrantLock + Condition	BlockingQueue
Level	Low Level	Medium Level	High Level
Lock Release	Automatic (on exit/exception)	Manual (finally { unlock() })	Internal
Wait Sets	One per Object	Multiple per Lock	Internal
Fairness	Non-fair (usually)	Configurable (new Lock(true))	Configurable
Interruptible	No (wait is interruptible, lock acquire is not)	Yes (lockInterruptibly)	Yes
Recommendation	Legacy / Simple cases	Complex coordination	Producer-Consumer

Conclusion & Recommendations

1. **Avoid Raw ITC:** Do not use wait/notify for new development unless you are building a low-level library.
2. **Use JUC:** Prefer BlockingQueue, ExecutorService, CompletableFuture, and ReentrantLock.
3. **Deadlock Prevention:** Enforce lock ordering. If using ReentrantLock, use tryLock with timeouts.
4. **Wait Loops:** Always wrap wait() or await() in a while loop checking the predicate.
5. **Diagnosis:** Learn to read Thread Dumps (jstack). It is the primary tool for debugging Deadlocks and Starvation in production.

Synchronous vs. Asynchronous Programming

1.1 Definitions

Synchronous Programming

- **Definition:** Execution happens sequentially. The caller **blocks** (waits) until the task is completed before moving to the next line of code.
- **Flow:** Start Task -> **Wait** -> Task Done -> Continue.
- **Analogy:** Ordering food at a counter. You order, stand there waiting, receive the food, then leave. You cannot do anything else while waiting.
- **Java Example:** Standard method calls, Thread.sleep(), InputStream.read().

// Synchronous

String data = readFromFile(); // Thread blocks here until I/O finishes

process(data); // Only runs after line above completes

1. Synchronous Programming

Concept: The main thread **waits** (blocks) until the task is done.

Observation: "Start" -> "Wait 2 seconds" -> "End".

```java
public class SyncExample {
    public static void main(String[] args) {
        System.out.println("1. Start");
        doHeavyTask(); // Main thread STOPS here until this finishes
        System.out.println("3. End");
    }

    private static void doHeavyTask() {
        try {
            Thread.sleep(2000); // Simulate work
            System.out.println("2. Task Completed");
        } catch (InterruptedException e) { e.printStackTrace(); }
    }
}
```

Output Order: 1 → 2 → 3

Asynchronous Programming

- **Definition:** Execution initiates a task and immediately continues without waiting. The result is handled later via a **callback**, **future**, or **event**.
- **Flow:** Start Task -> **Continue** -> (Task Done in background) -> Notify Result.
- **Analogy:** Ordering food for delivery. You order, hang up the phone, go watch TV. When food arrives, the doorbell rings (callback).
- **Java Example:** CompletableFuture, Callback interfaces, NIO (Non-blocking I/O).

// Asynchronous

CompletableFuture.supplyAsync(() -> readFromFile())

.thenAccept(data -> process(data)); // Runs when data is ready

// Code here executes immediately, without waiting for readFromFile

2. Asynchronous Programming

Concept: The main thread **continues** immediately. The task runs in the background.
Observation: "Start" → "End" → "Task Completed" (later).

```java
public class AsyncExample {
    public static void main(String[] args) {
        System.out.println("1. Start");

        // Run task in background thread
        CompletableFuture.runAsync(() -> {
            try { Thread.sleep(2000); } catch (InterruptedException e) {}
            System.out.println("2. Task Completed");
        });

        System.out.println("3. End"); // Does NOT wait for task

        // Keep main alive briefly to see async result
        try { Thread.sleep(3000); } catch (InterruptedException e) {}
    }
}
```

Output Order: 1 → 3 → 2

1.2 Comparison & Use Cases

Feature	Synchronous	Asynchronous
Code Simplicity	High (Linear logic)	Low (Callback hell, complex flow)
Resource Usage	High (One thread per request)	Low (Few threads handle many requests)
Blocking	Yes (Thread sleeps)	No (Thread remains free)
Error Handling	Try-Catch blocks	Callback error handlers / .exceptionally()
Best For	CPU-bound tasks, Simple logic	I/O-bound tasks (DB, Network), High concurrency

1.3 Internal Working: The Thread Model

- **Sync:** 1 Request = 1 Thread. If you have 10,000 concurrent users, you need 10,000 threads. Context switching kills performance.
- **Async:** 10,000 Requests can be handled by a small thread pool (e.g., 50 threads). When a thread hits an I/O wait, it yields the CPU to handle another request.

Part 2: Locks (Intrinsic vs. Explicit)

Before diving into specific locks, we must distinguish the two types of locking in Java.

2.1 Intrinsic Locks (synchronized)

- **Managed by:** JVM.
- **Acquisition:** Automatic upon entering block/method.
- **Release:** Automatic upon exit (normal or exception).
- **Limitations:** Cannot interrupt a thread waiting for a lock, no timeout, no fairness guarantee.

2.2 Explicit Locks (java.util.concurrent.locks.Lock)

- **Managed by:** Code (Developer).
- **Acquisition:** lock.lock().
- **Release:** lock.unlock() (**Must** be in finally block).
- **Advantages:**
    1. **Try Lock:** tryLock() (non-blocking attempt).
    2. **Interruptible:** lockInterruptibly() (wait for lock but respond to interrupt).
    3. **Fairness:** Can enforce FIFO ordering.
    4. **Multiple Conditions:** One lock can have multiple Condition objects (wait sets).

Part 3: ReentrantLock (Deep Dive)

3.1 Definition

A ReentrantLock is an implementation of the Lock interface that provides the same basic behavior as synchronized but with extended capabilities.

"Reentrant" means: If a thread already holds the lock, it can acquire it again without deadlocking itself. It maintains a **hold count**.

3.2 Internal Working: AQS (AbstractQueuedSynchronizer)

ReentrantLock is built on top of a framework called **AQS**. This is the engine behind most java.util.concurrent locks.

The Core Components of AQS

1. **State (volatile int state):**
    - 0: Lock is free.
    - 1+: Lock is held. The integer value represents the **Reentrancy Count**.
2. **FIFO Queue (CLH Queue):**
    - A doubly-linked list of Node objects.
    - Each Node represents a **Thread** waiting for the lock.
3. **CAS (Compare-And-Swap):**

- CPU-level atomic instruction used to update state.
- Ensures thread safety without blocking during the acquisition attempt.

Internal Pictorial Logic (Acquisition)

Internal Pictorial Logic (Acquisition)

```text
[ Thread A ] tries to lock()
      |
      v
+----------------------+
| 1. Check State (CAS) |
|    If state == 0     | ----(Success)----> Set state = 1, Set Owner = Thread A
|    Else              |
+----------------------+
      | (Fail: Lock held by Thread B)
      v
+----------------------+
| 2. Add to Wait Queue |
|    [Head] <-> [Node A]|
+----------------------+
      |
      v
+----------------------+
| 3. Park Thread       | (LockSupport.park())
|    Thread A sleeps   | (CPU usage = 0)
+----------------------+
```

Reentrancy Logic

If **Thread A** holds the lock (state = 1) and calls lock() again:

1. Check currentThread == owner. **True**.
2. Increment state to 2.
3. Return immediately (no queuing).
4. On unlock(), decrement state to 1.
5. On second unlock(), decrement state to 0 and release the lock.

3.3 Fairness vs. Non-Fairness

- **Non-Fair (Default):** When a lock is released, *any* waiting thread or a *new* arriving thread can grab it.
  - **Pros:** Higher throughput (less context switching).
  - **Cons:** Risk of **Starvation** (some threads wait forever).
- **Fair (new ReentrantLock(true)):** Threads acquire the lock in FIFO order (longest waiting first).
  - **Pros:** No starvation.
  - **Cons:** Lower throughput (more overhead managing the queue).

3.4 Use Cases

1. **Timeouts:** You need to try acquiring a lock for 5 seconds, then give up

```
if (lock.tryLock(5, TimeUnit.SECONDS)) {
   try { /* work */ } finally { lock.unlock(); }
} else {
   /* handle failure */
}
```

Example:

3. ReentrantLock (Explicit Lock)

Concept: Manual control over locking. Must unlock in `finally` to prevent deadlocks.
Key Feature: Shows `tryLock` (ability to give up if lock isn't available).

```java
public class ReentrantLockExample {
    private static final Lock lock = new ReentrantLock();
    private static int count = 0;

    public static void main(String[] args) {
        new Thread(() -> {
            if (lock.tryLock()) { // Try to acquire, don't wait forever
                try {
                    count++;
                    System.out.println("Count updated: " + count);
                } finally {
                    lock.unlock(); // MUST unlock manually
                }
            } else {
                System.out.println("Could not get lock, skipping...");
            }
        }).start();
    }
}
```

4: ReadWriteLock (Deep Dive)

4.1 Definition

A ReadWriteLock maintains a pair of associated locks:

1. **Read Lock:** Can be held simultaneously by **multiple** reader threads.
2. **Write Lock:** Exclusive. Only **one** writer thread can hold it, and no readers can be present.

Implementation: ReentrantReadWriteLock.

4.2 Theory: Why use it?

In many applications, data is **Read-Heavy** and **Write-Light** (e.g., a cache, a configuration manager).

- Using a standard ReentrantLock serializes *everything*. Readers wait for other readers unnecessarily.
- ReadWriteLock allows concurrent reads, increasing throughput.

4.3 Internal Working: Bit Manipulation

ReentrantReadWriteLock uses a single int state variable (32 bits) inside AQS to track both read and write locks to save memory and atomic operations.

State Structure (32-bit Integer)

```
| 16 Bits (High) | 16 Bits (Low) |
|----------------|---------------|
|  Read Count    |  Write Count  |
```

- **High 16 bits:** Track how many threads hold the **Read Lock**.
- **Low 16 bits:** Track the reentrancy count of the **Write Lock**.

Internal Pictorial Logic

State Variable: 0x00030001

Binary: 0000000000000011 0000000000000001

^ Read Count = 3 ^ Write Count = 1

Lock Compatibility Matrix

Current State	Request	Result
No Lock	Read	**Grant**
No Lock	Write	**Grant**
Read Lock(s)	Read	**Grant** (Increment Read Count)
Read Lock(s)	Write	**Block** (Wait for readers to finish)
Write Lock	Read	**Block** (Wait for writer to finish)
Write Lock	Write	**Grant** (If same thread, Reentrant)

4.4 Lock Upgrading and Downgrading

- **Downgrading (Allowed):** Acquire Write Lock -> Acquire Read Lock -> Release Write Lock.
  - *Why:* You updated data (Write) and now want to continue reading (Read) while allowing other readers in.
- **Upgrading (NOT Allowed):** Acquire Read Lock -> Try to acquire Write Lock.
  - *Why:* **Deadlock Risk.**
  - *Scenario:* Thread A holds Read. Thread B waits for Write. Thread A waits for Write (Upgrade).

- Thread B cannot proceed because Thread A holds Read. Thread A cannot proceed because Thread B is in the queue (depending on fairness) or simply because Write lock is exclusive.
- *Result:* Deadlock.

4.5 Use Cases

```
ReadWriteLock rwLock = new ReentrantReadWriteLock();


public Data get(String key) {
  rwLock.readLock().lock();
  try { return cache.get(key); }
  finally { rwLock.readLock().unlock(); }
}


public void put(String key, Data val) {
  rwLock.writeLock().lock(); // Exclusive
  try { cache.put(key, val); }
  finally { rwLock.writeLock().unlock(); }
}
```

4. ReadWriteLock

Concept: Multiple threads can **Read** at the same time, but **Write** is exclusive.
Key Feature: Shows separate locks for reading and writing.

```java
public class ReadWriteLockExample {
    private static final ReadWriteLock rwLock = new ReentrantReadWriteLock();
    private static String data = "Initial";

    public static void main(String[] args) {
        // Reader 1
        new Thread(() -> {
            rwLock.readLock().lock();
            try { System.out.println("Reader 1 sees: " + data); }
            finally { rwLock.readLock().unlock(); }
        }).start();

        // Reader 2 (Can run AT THE SAME TIME as Reader 1)
        new Thread(() -> {
            rwLock.readLock().lock();
            try { System.out.println("Reader 2 sees: " + data); }
            finally { rwLock.readLock().unlock(); }
        }).start();

        // Writer (Must wait for BOTH readers to finish)
        new Thread(() -> {
            rwLock.writeLock().lock();
            try { data = "Updated"; System.out.println("Data Updated"); }
            finally { rwLock.writeLock().unlock(); }
        }).start();
    }
}
```

1. **Configuration Managers:** Many threads read config, only one thread updates it periodically.

4.6 Modern Alternative: StampedLock

Introduced in Java 8.

- **Optimistic Reading:** Allows reading *without* locking first. You check a "stamp" (version number) after reading. If the stamp changed, a write occurred, so you retry with a read lock.
- **Performance:** Faster than ReadWriteLock for extremely read-heavy workloads.
- **Risk:** More complex to use correctly.

Part 5: Summary & Comparison

5.1 Lock Comparison Table

Feature	synchronized	ReentrantLock	ReentrantReadWriteLock
Type	Intrinsic (JVM)	Explicit (API)	Explicit (API)
Unlock	Automatic	Manual (finally)	Manual (finally)
Reentrant	Yes	Yes	Yes
Fairness	No	Optional (true)	Optional (true)
Timeout	No	Yes (tryLock)	Yes (tryLock)
Interruptible	No	Yes	Yes
Concurrency	Exclusive	Exclusive	Shared (Read) / Exclusive (Write)
Conditions	1 per lock	Multiple	Multiple per lock type
Performance	Improved in Java 6+	High (under contention)	Highest (Read-Heavy)

5.2 Internal Architecture Diagram (Textual)

```text
+-------------------------------------------------------------+
|                    java.util.concurrent                     |
+-------------------------------------------------------------+
|  ReentrantLock          ReentrantReadWriteLock              |
|       |                       |                             |
|       +------------+----------+                             |
|                    |                                        |
|          AbstractQueuedSynchronizer (AQS)                   |
|                    |                                        |
|      +-------------+------------+                           |
|      |                          |                           |
| [ volatile int state ]    [ FIFO Wait Queue ]               |
|      |                          |                           |
|      +-------------+------------+                           |
|                    |                                        |
|           LockSupport.park()/unpark()                       |
|           (Wraps OS Native Mutex/Pthread)                   |
+-------------------------------------------------------------+
```

5.3 Best Practices & Pitfalls

1. Always Unlock in Finally:

```java
lock.lock();
try {
    // critical section
} finally {
    lock.unlock(); // CRITICAL: Prevents deadlock if exception occurs
}
```

1. **Avoid Locking I/O:** Never hold a lock while performing network calls or disk I/O. It blocks other threads unnecessarily.
2. **Prefer ReentrantLock over synchronized when:**
    - You need timeouts.
    - You need interruptible lock waits.
    - You need multiple condition variables.
3. **Prefer ReadWriteLock when:**
    - Reads >> Writes.
    - Data consistency is critical during reads.
4. **Watch out for Write Starvation in ReadWriteLock:** If reads are continuous, a writer might never get a chance to lock. (Use StampedLock or fair locks to mitigate).

5.4 When to use what?

- **Simple synchronization:** synchronized.
- **Complex coordination / Timeouts:** ReentrantLock.
- **Read-Heavy Data Structures:** ReentrantReadWriteLock or StampedLock.
- **Producer-Consumer:** BlockingQueue (Avoid manual locks).
- **One-time events:** CountDownLatch / CyclicBarrier.

Executor service

the **Java Concurrency Utilities (java.util.concurrent)** framework, specifically focusing on the Executor framework, Thread Pools, and Future/Callable patterns.

Part 1: The Executor Hierarchy

1.1 The Problem: Raw Threads

Creating a new Thread for every task is expensive (memory overhead, CPU context switching). Unbounded thread creation leads to **OutOfMemoryError** and system crashes. **Solution:** Reuse existing threads via a **Thread Pool**.

1.2 The Interface Hierarchy

1. Executor (The Base)

- **Definition:** An interface with a single method execute(Runnable command).
- **Purpose:** Decouples task submission from task execution.
- **Limitation:** No lifecycle management (cannot shutdown gracefully).

2. ExecutorService (The Manager)

- **Definition:** Subinterface of Executor.
- **Purpose:** Adds lifecycle management (shutdown(), awaitTermination()) and asynchronous result handling (submit()).
- **Key Methods:**
    - submit(Callable): Returns a Future.
    - invokeAll(): Executes a collection of tasks.
    - shutdown(): Stops accepting new tasks, finishes existing ones.

3. ThreadPoolExecutor (The Engine)

- **Definition:** The concrete implementation of ExecutorService.
- **Purpose:** Configurable thread pool with control over core size, max size, queue type, and rejection policies.
- **Note:** Avoid Executors.newFixedThreadPool() in production (uses unbounded queue). Prefer explicit ThreadPoolExecutor constructors.

1. execute() vs. submit()

These are the two primary methods to send tasks to a thread pool. They look similar but behave differently regarding **return values** and **exception handling**.

Feature	execute(Runnable)	submit(Callable/Runnable)
Origin	Executor interface (Base)	ExecutorService interface
Return Type	void (Nothing)	Future<?> (Placeholder for result)
Exceptions	Printed to System.err (Uncaught)	Wrapped in ExecutionException (Retrievable via get())
Use Case	Fire-and-forget tasks	Tasks needing results or error handling

Internal Difference

- **execute**: Passes the Runnable directly to the ThreadPoolExecutor. If the task throws an exception, the Thread's UncaughtExceptionHandler catches it.
- **submit**: Wraps your Runnable/Callable inside a **FutureTask**. The FutureTask catches the exception and stores it. You only see the exception if you call future.get().

---

2. Types of ExecutorServices

Java provides factory methods in the Executors class. **Warning:** Most of these have production risks.

1. newFixedThreadPool(int n)

- **What:** Pool with a fixed number of threads.
- **Queue: Unbounded** LinkedBlockingQueue.
- **Risk: OutOfMemoryError**. If tasks arrive faster than they are processed, the queue grows infinitely.
- **Use:** When you know the exact load and it won't spike.

2. newCachedThreadPool()

- **What:** Creates new threads as needed, reuses idle ones.
- **Queue: SynchronousQueue** (Handoff queue, capacity 0).
- **Risk: System Crash**. If load spikes, it creates unlimited threads (up to OS limit).
- **Use:** Short-lived, asynchronous tasks with low overall load.

3. newSingleThreadExecutor()

- **What:** Exactly one worker thread.
- **Queue:** Unbounded LinkedBlockingQueue.
- **Use:** Ensuring tasks execute sequentially (FIFO).

4. newScheduledThreadPool(int n)

- **What:** Supports delayed and periodic execution.

- **Use:** Cron jobs, heartbeats, recurring tasks.

5. newWorkStealingPool() (Java 8+)

- **What:** Uses ForkJoinPool. Idle threads "steal" tasks from busy threads.
- **Use:** CPU-intensive tasks where sub-tasks can be broken down.

---

3. Core Pool Size vs. Maximum Pool Size (The Logic)

This is the most misunderstood part of ThreadPoolExecutor. The interaction between **Core**, **Queue**, and **Max** follows a specific order.

The Execution Flow Algorithm

When execute(task) is called:

1. **Check Core:** If runningThreads < corePoolSize, create a **new Core Thread**.
2. **Check Queue:** If runningThreads >= corePoolSize, try to **add to Queue**.
3. **Check Max:** If Queue is **FULL** and runningThreads < maximumPoolSize, create a **new Non-Core Thread**.
4. **Reject:** If Queue is **FULL** and runningThreads == maximumPoolSize, trigger **Rejection Policy**.

Pictorial Flow

Pictorial Flow

text

```text
Task Arrives
    |
    v
[ Is Thread Count < Core Size? ] --YES--> Create New Core Thread --> Run Task
    | NO
    v
[ Is Queue Full? ] -------------NO-----> Add to Queue --> Wait for Thread
    | YES
    v
[ Is Thread Count < Max Size? ] --YES--> Create New Max Thread --> Run Task
    | NO
    v
[ Trigger Rejection Policy ]
```

Critical Insight

- **Max Pool Size is ignored if the Queue is Unbounded.** (e.g., `newFixedThreadPool`). The queue will fill up be
  the pool ever grows to `maximumPoolSize`.
- **To use Max Pool Size effectively, you MUST use a Bounded Queue** (e.g., `ArrayBlockingQueue`).

Part 2: Tasks and Results (Callable & Future)

2.1 Runnable vs. Callable

Feature	Runnable (Legacy)	Callable (Modern)
Method	run()	call()

Feature	Runnable (Legacy)	Callable (Modern)
Return Value	void (None)	Generic T (Result)
Exceptions	Cannot throw checked exceptions	Can throw checked exceptions
Usage	executor.execute(r)	executor.submit(c)

2.2 Future (The Placeholder)

- **Definition:** Represents the **pending result** of an asynchronous computation.
- **Analogy:** A receipt given to you when you drop off clothes at a dry cleaner. You don't have the clothes (result) yet, but the receipt (Future) allows you to check status or pick them up later.
- **Key Methods:**
  - get(): Blocks until result is ready.
  - get(timeout, unit): Blocks with timeout.
  - isDone(): Checks if computation finished.
  - cancel(mayInterrupt): Attempts to stop execution.

2.3 Internal Working: FutureTask

When you submit a Callable, the Executor wraps it in a FutureTask.

- FutureTask implements both Runnable and Future.
- It uses **CAS (Compare-And-Swap)** to manage state (NEW, COMPLETING, NORMAL, EXCEPTIONAL, CANCELLED).
- When get() is called, if the state is not complete, the calling thread parks (waits) on an internal LockSupport until the worker thread sets the result and unparks the waiter.

Part 3: ThreadPoolExecutor (Deep Dive)

3.1 Constructor Parameters

java

```
new ThreadPoolExecutor(
  int corePoolSize,       // Min threads kept alive
  int maximumPoolSize,    // Max threads allowed
  long keepAliveTime,     // Time excess threads wait for new tasks
  TimeUnit unit,          // Time unit
  BlockingQueue<Runnable> workQueue, // Queue for holding tasks
```

```
    ThreadFactory threadFactory,      // Custom thread naming/priority
    RejectionHandler handler        // What to do when full
)
```

Key Implementation Parameters

- **corePoolSize**: The number of threads to keep in the pool.
- **maximumPoolSize**: The maximum number of threads allowed.
- **keepAliveTime**: Idle threads wait this long before terminating.
- **workQueue**: The queue used to hold tasks before they are executed.
- **RejectedExecutionHandler**: Policy for tasks that cannot be executed.

Example

```
ExecutorService executor = Executors.newFixedThreadPool(10);
// Or for customized control:
ExecutorService customExecutor = new ThreadPoolExecutor(
    2, 4, 60L, TimeUnit.SECONDS, new LinkedBlockingQueue<Runnable>()
);
```

3.2 The Execution Flow (The Logic Tree)

This is the most critical internal logic. When `execute(task)` is called:

```text
1. Is current worker count < corePoolSize?
   ├── YES → Create new Thread to run task immediately.
   └── NO  → Go to 2.

2. Can task be added to workQueue? (queue.offer(task))
   ├── YES → Task sits in queue. Worker threads pick it up later.
   └── NO  → (Queue is Full) → Go to 3.

3. Is current worker count < maximumPoolSize?
   ├── YES → Create new Thread (beyond core size) to run task.
   └── NO  → (Pool is Saturated) → Go to 4.

4. Execute Rejection Policy.
```

Note: This explains why `corePoolSize` threads are created *before* the queue fills up, but `maximumPoolSize` threads are only created *after* the queue is full.

3.3 Internal Architecture: ctl and Worker

1. The ctl Variable

ThreadPoolExecutor uses a single AtomicInteger ctl to track two things simultaneously to ensure atomicity:

- **High 3 bits:** Run State (RUNNING, SHUTDOWN, STOP, TIDYING, TERMINATED).
- **Low 29 bits:** Worker Count (Number of active threads).
- **Why?** Allows checking state and count in one atomic CAS operation.

2. The Worker Class

- Threads in the pool are wrapped in Worker objects.
- Worker implements Runnable.

The threads in the pool are not raw `Thread` objects; they are wrapped in `Worker` objects (inner class of `ThreadPoolExecutor`).

- `Worker` implements `Runnable`.
- It holds the actual `Thread` object.
- It manages the lock state (to prevent interruption while running a task).

- **Internal Loop**

```
public void run() {
  Runnable task = firstTask;
  while (task != null || (task = getTask()) != null) {
    runTask(task);
    task = null; // Allow core threads to timeout if needed
  }
}
```

- getTask() pulls from the BlockingQueue. If the queue is empty, the thread **parks** (sleeps) until enqueue signals it.

This method pulls tasks from the `BlockingQueue`.

- **Core Threads:** If `allowCoreThreadTimeOut` is false, they block **indefinitely** waiting for a task.
- **Non-Core Threads:** They block for `keepAliveTime`. If no task arrives in that time, `getTask()` returns `null`, the loop exits, and the thread dies (pool shrinks).
-

3.4 Pictorial Representation of Thread Pool

```text
+-----------------------+
|   Client / Main App   |
+----------+------------+
           | submit(task)
           v
+-----------------------+
|   ThreadPoolExecutor  |
|                       |
|  +-----------------+  |
|  |   Task Queue    |  | <--- BoundedQueue (e.g., capacity 10)
|  | [T1][T2][T3]... |  |
|  +--------+--------+  |
           |            |
           | (Workers poll queue)
           v            v
+----------------------+----------------------+
|   Worker Thread 1    |   Worker Thread 2    |
|   (Core)             |   (Core)             |
|   [ Running Task ]   |   [ Waiting (Park) ] |
+----------------------+----------------------+
           ^
           | (If Queue Full & Count < Max)
           |
+-----------------------+
|   Worker Thread 3     |
|   (Extra/Max)         |
+-----------------------+
```

5. Shutdown Mechanisms

ExecutorService has two shutdown methods with distinct behaviors.

1. shutdown() (Graceful)

- **State:** RUNNING → SHUTDOWN.
- **Behavior:**
    1. Stop accepting **new** tasks.
    2. Allow **queued** tasks to execute.
    3. Allow **running** tasks to finish.
- **Analogy:** "Stop taking new orders, finish cooking the food in the queue, then close the kitchen."

2. shutdownNow() (Forceful)

- **State:** RUNNING → STOP.
- **Behavior:**
    1. Stop accepting new tasks.
    2. **Clear the Queue** (returns list of unstarted tasks).
    3. **Interrupt** running threads (sends InterruptedException).
- **Analogy:** "Turn off the lights, throw away uncooked food, and tell cooks to stop immediately."

3. awaitTermination(timeout)

- Blocks the calling thread until the pool terminates or the timeout expires. Used after shutdown() to wait for cleanup.

Part 4: Rejection Policies

When the Pool is saturated (Queue Full + Max Threads Reached), the RejectedExecutionHandler is triggered.

4.1 The 4 Built-in Policies

Policy	Behavior	Use Case
AbortPolicy (Default)	Throws RejectedExecutionException.	**Safety.** Fail fast. Notify caller that system is overloaded.
CallerRunsPolicy	The **submitting thread** runs the task itself.	**Throttling.** Slows down the producer automatically. Good for load shedding.
DiscardPolicy	Silently drops the task. No exception.	**Logging/Metrics.** If losing some data points is acceptable (e.g., monitoring metrics).
DiscardOldestPolicy	Drops the *oldest* task in the queue, retries submitting new one.	**Priority.** Keep newest data, lose old pending data.

4.2 Custom Policy Example

```
new ThreadPoolExecutor(..., new RejectedExecutionHandler() {
  public void rejectedExecution(Runnable r, ThreadPoolExecutor e) {
    // Log to DB, send alert, or save to disk for later retry
    System.out.println("Task rejected! Saving to dead-letter queue.");
  }
});
```

Best Practices & Pitfalls

5.1 The Executors Factory Warning

DO NOT USE Executors.newFixedThreadPool() or Executors.newCachedThreadPool() in production.

- **Reason:** newFixedThreadPool uses an **unbounded** LinkedBlockingQueue (Integer.MAX_VALUE).
- **Risk:** If tasks arrive faster than they are processed, the queue grows until **OutOfMemoryError**.
- **Solution:** Always instantiate ThreadPoolExecutor directly with a **bounded** queue (e.g., ArrayBlockingQueue).

5.2 Setting Thread Names

Always use a custom `ThreadFactory`. Debugging thread dumps with names like `pool-1-thread-5` is hard.

```java
ThreadFactory factory = r -> {
    Thread t = new Thread(r);
    t.setName("OrderProcessor-" + t.getId());
    return t;
};
```

5.3 Graceful Shutdown

Never let the JVM kill threads abruptly.

```java
executor.shutdown(); // Stop accepting new tasks
try {
    if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
        executor.shutdownNow(); // Force kill if not done
    }
} catch (InterruptedException e) {
    executor.shutdownNow();
}
```

Summary Checklist

Concept	Key Takeaway
Executor	Use ExecutorService interface, implement with ThreadPoolExecutor.
Runnable vs Callable	Use Callable if you need a return value or exception handling.
Future	Use Future.get() to retrieve results; beware it blocks.
Queue	**Always** use a bounded queue (ArrayBlockingQueue) to prevent OOM.
Rejection	Use CallerRunsPolicy for backpressure, AbortPolicy for safety.
Shutdown	Always call shutdown() and awaitTermination() in application exit hooks.
Internal	Understand Core vs Max pool logic (Core -> Queue -> Max -> Reject).

6: Complete Code Example

This example combines Callable, Future, ThreadPoolExecutor, and RejectionPolicy.

```java
public class ExecutorDeepDive {
    public static void main(String[] args) {
        // 1. Define Bounded Queue (Critical for production)
        BlockingQueue<Runnable> queue = new ArrayBlockingQueue<>(5);

        // 2. Create ThreadPoolExecutor explicitly
        ThreadPoolExecutor executor = new ThreadPoolExecutor(
            2,                      // Core Pool (2 threads always alive)
            4,                      // Max Pool (Grow to 4 if queue full)
            60L,                    // Keep Alive Time
            TimeUnit.SECONDS,
            queue,                  // Bounded Queue
            new ThreadPoolExecutor.CallerRunsPolicy() // Throttle producer if full
        );

        List<Future<Integer>> futures = new ArrayList<>();

        // 3. Submit Callable Tasks
        for (int i = 1; i <= 10; i++) {
            int taskId = i;
            Future<Integer> future = executor.submit(() -> {
                Thread.sleep(1000); // Simulate work
                return taskId * 2;  // Return result
            });
            futures.add(future);
            System.out.println("Submitted Task: " + taskId);
        }

        // 4. Process Results
        for (Future<Integer> f : futures) {
            try {
                System.out.println("Result: " + f.get()); // Blocks until ready
            } catch (Exception e) { e.printStackTrace(); }
        }

        // 5. Shutdown
        executor.shutdown();
    }
}
```

ecution Flow of Example:

. **Tasks 1-2:** Assigned to Core Threads immediately.
. **Tasks 3-7:** Queue fills up (Capacity 5).
. **Tasks 8-9:** Queue full. Pool grows to Max (4 threads). Tasks 8 & 9 run on new threads.
. **Task 10:** Queue full + Max threads reached. `CallerRunsPolicy` triggers. The **Main Thread** executes Task 10 synchronously.
. **Results:** Main thread collects results via `f.get()`.

Part 1: Callable and Future API

1.1 Definition & Concepts

Callable<V>: A task that returns a result and may throw an exception.

- Functional interface with single method: V call() throws Exception
- Unlike Runnable.run(), it returns a value and can throw checked exceptions.

Future<V>: Represents the result of an asynchronous computation.

- Provides methods to check if computation is complete, wait for completion, and retrieve the result.

1.2 Internal Working: FutureTask

When you submit a Callable to an executor, it's wrapped in a

```java
// Simplified internal structure
public class FutureTask<V> implements RunnableFuture<V> {
    private volatile int state; // NEW, COMPLETING, NORMAL, EXCEPTIONAL, CANCELLED
    private Callable<V> callable;
    private Object outcome; // result or exception
    private volatile Thread runner;
    private WaitNode waiters; // Linked list of waiting threads

    public V get() throws InterruptedException, ExecutionException {
        if (state != NORMAL) {
            awaitDone(false, 0L); // Park thread until complete
        }
        return report(outcome);
    }

    public void run() {
        if (state != NEW) return;
        runner = Thread.currentThread();
        try {
            V result = callable.call();
            outcome = result;
            state = NORMAL;
        } catch (Throwable ex) {
            outcome = ex;
            state = EXCEPTIONAL;
        } finally {
            finishCompletion(); // Wake up all waiting threads
        }
    }
}
```

/ Internal Mechanisms:

State Management: Uses CAS (Compare-And-Swap) for thread-safe state transitions
Waiter Queue: Threads calling `get()` are added to a wait queue and parked using `LockSupport.park()`
Completion: When task finishes, all waiting threads are unparked using `LockSupport.unpark()`

1.3 Methods

Callable:

- V call() throws Exception - Main method

Future:

- boolean cancel(boolean mayInterruptIfRunning) - Cancel task
- boolean isCancelled() - Check if cancelled
- boolean isDone() - Check if completed
- V get() - Block until result available
- V get(long timeout, TimeUnit unit) - Block with timeout

1.4 When to Use

- **Use Callable**: When task needs to return a value or throw checked exceptions
- **Use Future**: When you need to retrieve result later or check task status

1.5 Simple Example

public class CallableFutureExample {

  public static void main(String[] args) throws Exception {

```
    ExecutorService executor = Executors.newFixedThreadPool(2);
        // Submit Callable task
    Future<Integer> future = executor.submit(() -> {
      Thread.sleep(1000);
      return 42; // Returns result
    });
        // Do other work...
    System.out.println("Waiting for result...");
        // Get result (blocks until available)
    Integer result = future.get();
    System.out.println("Result: " + result);
        executor.shutdown();
  }
}
```

Part 2: Synchronizers

2.1 CountDownLatch

Definition

A synchronization aid that allows one or more threads to wait until a set of operations being performed in other threads completes.

Key Characteristic: **One-time use** - count cannot be reset.

Internal Working

```
public class CountDownLatch {
  private static final class Sync extends AbstractQueuedSynchronizer {
    Sync(int count) {
      setState(count);
    }
       int getCount() { return getState(); }
        protected int tryReleaseShared(int releases) {
      for (;;) {
        int c = getState();
```

```
            if (c == 0) return 0;
            int nextc = c - 1;
            if (compareAndSetState(c, nextc))
              return nextc == 0 ? 1 : 0;
        }
    }
            protected int tryAcquireShared(int acquires) {
        return getState() == 0 ? 1 : -1;
    }
  }
    private final Sync sync;
    public void await() throws InterruptedException {
    sync.acquireSharedInterruptibly(1);
  }
    public void countDown() {
    sync.releaseShared(1);
  }
}
```

Internal Mechanism:

- Uses AQS (AbstractQueuedSynchronizer) with shared mode
- State = count
- countDown(): Decrements state using CAS, when reaches 0, releases all waiting threads
- await(): Blocks if state > 0

Methods

- await() - Block until count reaches zero
- await(long timeout, TimeUnit unit) - Block with timeout
- countDown() - Decrement count
- getCount() - Current count

When to Use

- Start multiple threads and wait for all to complete

- Service initialization wait
- **One-time event** synchronization

Example

```
public class CountDownLatchExample {
  public static void main(String[] args) throws InterruptedException {
    CountDownLatch latch = new CountDownLatch(3);
        for (int i = 0; i < 3; i++) {
      new Thread(() -> {
        System.out.println(Thread.currentThread().getName() + " working...");
        try { Thread.sleep(1000); } catch (InterruptedException e) {}
        latch.countDown();
        System.out.println(Thread.currentThread().getName() + " done");
      }).start();
    }
        System.out.println("Main waiting for workers...");
    latch.await(); // Block until count = 0
    System.out.println("All workers completed!");
  }
}
```

2.2 CyclicBarrier

Definition

A synchronization aid that allows a set of threads to all wait for each other to reach a common barrier point.

Key Characteristic: **Reusable** - can be used again after barrier is reached.

Internal Working

Internal Working

```java
public class CyclicBarrier {
    private static class Generation {
        boolean broken = false;
    }

    private final ReentrantLock lock = new ReentrantLock();
    private final Condition trip = lock.newCondition();
    private final int parties;
    private int count;
    private Generation generation = new Generation();

    private void nextGeneration() {
        trip.signalAll(); // Wake all threads
        count = parties;
        generation = new Generation();
    }

    private int dowait(boolean timed, long nanos) {
        lock.lock();
        try {
            Generation g = generation;
            count--;
            if (count == 0) { // All parties arrived
                nextGeneration();
                return 0;
            }
            // Wait for signal
            for (;;) {
                try {
                    trip.await();
                    if (g.broken) throw new BrokenBarrierException();
                    if (g != generation) return 0;
                } catch (InterruptedException ie) {
                    g.broken = true;
                    throw ie;
                }
            }
        } finally {
            lock.unlock();
        }
    }
}
```

Internal Mechanism:

- Uses ReentrantLock and Condition
- Count tracks arrivals
- When count reaches 0, signals all waiting threads and resets
- Generation object tracks barrier state (broken/active)

Methods

- await() - Wait until all parties arrive
- await(long timeout, TimeUnit unit) - Wait with timeout
- reset() - Reset barrier
- getNumberWaiting() - Number of threads waiting
- isBroken() - Check if barrier is broken

When to Use

- Parallel iterative algorithms (each iteration waits for all)

- Multi-phase computations
- **Reusable** synchronization point

Simple Example

```java
public class CyclicBarrierExample {
    public static void main(String[] args) {
        CyclicBarrier barrier = new CyclicBarrier(3, () ->
            System.out.println("All threads reached barrier!")
        );

        for (int i = 0; i < 3; i++) {
            new Thread(() -> {
                try {
                    System.out.println(Thread.currentThread().getName() + " phase 1");
                    Thread.sleep(1000);
                    barrier.await(); // Wait for all
                    System.out.println(Thread.currentThread().getName() + " phase 2");
                } catch (Exception e) {}
            }).start();
        }
    }
}
```

2.3 Semaphore

Definition

A counting semaphore that maintains a set of permits. Used to control access to a resource pool.

Key Characteristic: Controls **number of concurrent accesses**.

Internal Working

```java
public class Semaphore {
    private static final class Sync extends AbstractQueuedSynchronizer {
        Sync(int permits) { setState(permits); }
        final int getPermits() { return getState(); }

        protected int tryAcquireShared(int acquires) {
            for (;;) {
                int available = getState();
                int remaining = available - acquires;
                if (remaining < 0 || compareAndSetState(available, remaining))
                    return remaining;
            }
        }

        protected boolean tryReleaseShared(int releases) {
            for (;;) {
                int p = getState();
                if (compareAndSetState(p, p + releases))
                    return true;
            }
        }
    }

    public void acquire() throws InterruptedException {
        sync.acquireSharedInterruptibly(1);
    }

    public void release() {
        sync.releaseShared(1);
    }
}
```

Internal Mechanism:

- State = number of available permits
- acquire(): Decrements permits (blocks if 0)
- release(): Increments permits
- Uses AQS shared mode with FIFO queue

Methods

- acquire() - Acquire one permit (block if none)
- acquire(int permits) - Acquire multiple permits
- tryAcquire() - Try without blocking
- release() - Release permit
- availablePermits() - Available count
- drainPermits() - Get all permits

When to Use

- Limit concurrent database connections
- Rate limiting
- Resource pool management (e.g., 5 licenses for 10 users)

<u>Simple example</u>

```
public class SemaphoreExample {
  static Semaphore semaphore = new Semaphore(2); // Only 2 concurrent
   public static void main(String[] args) {
    for (int i = 0; i < 5; i++) {
      new Thread(() -> {
        try {
          semaphore.acquire();
          System.out.println(Thread.currentThread().getName() + " acquired");
          Thread.sleep(2000);
        } catch (InterruptedException e) {}
        finally {
          System.out.println(Thread.currentThread().getName() + " releasing");
          semaphore.release();
```

```
            }
        }).start();
    }
  }
}
```

2.4 Phaser

Definition

A reusable synchronization barrier that supports dynamic number of parties and multiple phases.

Key Characteristic: **Most flexible** - can register/deregister parties dynamically.

Internal Working

```java
public class Phaser {
    private volatile long state;
    // state layout:
    // bits 0-15: number of registered parties
    // bits 16-31: number of arrived parties
    // bits 32-63: phase number

    public int register() {
        return doRegister(1);
    }

    public int arriveAndAwaitAdvance() {
        return doArrive(1, true);
    }

    private int doArrive(int arrivals, boolean await) {
        for (;;) {
            long s = state;
            int phase = (int)(s >>> PHASE_SHIFT);
            int parties = (int)(s & PARTIES_MASK) >>> PARTIES_SHIFT;
            int arrived = (int)(s & ARRIVED_MASK) >>> ARRIVED_SHIFT;

            if (arrived + arrivals >= parties) {
                // Advance to next phase
                long next = ((long)(phase + 1) << PHASE_SHIFT) | PARTIES_MASK;
                if (compareAndSetState(s, next)) {
                    return phase;
                }
            } else {
                // Just increment arrived count
                long next = s + (arrivals << ARRIVED_SHIFT);
                if (compareAndSetState(s, next)) {
                    if (!await) return phase;
                    awaitAdvance(phase);
                }
            }
        }
    }
}
```

Internal Mechanism:

- Single `long` state variable packing: parties, arrived, phase
- CAS operations for thread safety
- Dynamic registration/deregistration
- Automatic phase advancement

Methods

- register() - Register new party
- bulkRegister(int parties) - Register multiple

- arrive() - Signal arrival, don't wait
- arriveAndAwaitAdvance() - Signal and wait
- arriveAndDeregister() - Signal and remove self
- awaitAdvance(int phase) - Wait for phase change
- getPhase() - Current phase
- getRegisteredParties() - Count

When to Use

- Multi-phase algorithms with dynamic participants
- When parties can join/leave during execution
- Complex coordination scenarios

Example

```
public class PhaserExample {
  public static void main(String[] args) {
    Phaser phaser = new Phaser(1); // Main thread registers
        for (int i = 0; i < 3; i++) {
      phaser.register();
      new Thread(() -> {
        try {
          System.out.println(Thread.currentThread().getName() + " phase 0");
          phaser.arriveAndAwaitAdvance();
          System.out.println(Thread.currentThread().getName() + " phase 1");
          phaser.arriveAndDeregister();
        } catch (Exception e) {}
      }).start();
    }
        phaser.arriveAndAwaitAdvance(); // Phase 0 done
    System.out.println("Phase 0 complete");
    phaser.arriveAndAwaitAdvance(); // Phase 1 done
    System.out.println("All done");
  }}
```

Part 3: Fork/Join Framework

3.1 Definition & Concepts

Fork/Join Framework: Designed for parallel execution of tasks that can be recursively broken into smaller subtasks.

Core Components:

- **ForkJoinPool**: Specialized ExecutorService for fork/join tasks
- **RecursiveTask<V>**: Task that returns a result
- **RecursiveAction**: Task that doesn't return result
- **Work-Stealing**: Idle threads steal tasks from busy threads

3.2 Work-Stealing Algorithm

3.2 Work-Stealing Algorithm

Internal Working

```java
// Each thread has a deque (double-ended queue)
// Owner thread: pushes/pops from top (LIFO)
// Thief thread: steals from bottom (FIFO)

public class WorkStealing {
    // Thread 1's deque: [A, B, C, D]
    // Thread 1 (owner): pop() -> D (top, LIFO - good for cache locality)
    // Thread 2 (thief): steal() -> A (bottom, FIFO - older tasks)

    // Benefits:
    // 1. LIFO for owner: recently created tasks likely in cache
    // 2. FIFO for thieves: steal older, larger tasks
    // 3. Lock-free for owner operations
    // 4. Minimal contention
}
```

Internal Architecture:

```
ForkJoinPool
    ├─ Worker Thread 1: Deque [T1, T2, T3]
    ├─ Worker Thread 2: Deque [T4, T5] (idle, steals from T1)
    ├─ Worker Thread 3: Deque [T6]
    └─ Common Pool (Java 8+)
```

3.5 Methods

ForkJoinTask:

- fork() - Schedule async execution (like submit)
- join() - Wait and get result (like get)
- compute() - Main computation (override this)

ForkJoinPool:

- execute(ForkJoinTask) - Execute async
- invoke(ForkJoinTask) - Execute and wait
- invokeAll(task1, task2) - Execute both, wait

3.6 When to Use

- **Use Fork/Join**: CPU-intensive, divide-and-conquer algorithms
- **Don't Use**: I/O-bound tasks (use regular thread pool)
- **Threshold**: Tasks should be small enough to parallelize but not too small (overhead)

3.3 ForkJoinPool Internal Working

```java
public class ForkJoinPool extends AbstractExecutorService {
    static final class WorkQueue {
        ForkJoinTask<?>[] array;
        int base; // Bottom (steal from here)
        int top;  // Top (owner pops from here)
        ForkJoinWorkerThread owner;

        // Owner pushes/pops at top
        void push(ForkJoinTask<?> t) {
            array[top++ & mask] = t;
        }

        ForkJoinTask<?> pop() {
            if (top > base) {
                return array[--top & mask];
            }
            return null;
        }

        // Thief steals from base
        ForkJoinTask<?> steal() {
            if (base < top) {
                ForkJoinTask<?> t = array[base & mask];
                base++;
                return t;
            }
            return null;
        }
    }

    // Pool manages array of WorkQueues (one per thread)
}
```

3.4 RecursiveTask vs RecursiveAction

Feature	RecursiveTask<V>	RecursiveAction
Returns	Value of type V	void
Method	`V compute()`	`void compute()`
Use When	Task produces result	Task performs side effects

3.5 Methods

Simple example

```
public class ForkJoinExample {

  // RecursiveTask: Returns result
  static class SumTask extends RecursiveTask<Long> {
    private final long[] array;
    private final int start, end;
    private static final int THRESHOLD = 1000;
```

```
SumTask(long[] array, int start, int end) {
    this.array = array;
    this.start = start;
    this.end = end;
}

@Override
protected Long compute() {
    int length = end - start;

    if (length <= THRESHOLD) {
        // Sequential computation
        long sum = 0;
        for (int i = start; i < end; i++) {
            sum += array[i];
        }
        return sum;
    }

    // Divide
    int mid = start + length / 2;
    SumTask left = new SumTask(array, start, mid);
    SumTask right = new SumTask(array, mid, end);

    // Fork (async)
    left.fork();
    // Compute right directly
    long rightResult = right.compute();
    // Join left
    long leftResult = left.join();
```

```
        return leftResult + rightResult;
    }
  }

  public static void main(String[] args) {
    long[] array = new long[10000];
    Arrays.fill(array, 1);

    ForkJoinPool pool = new ForkJoinPool();
    SumTask task = new SumTask(array, 0, array.length);

    long result = pool.invoke(task);
    System.out.println("Sum: " + result);

    pool.shutdown();
  }
}
```

Part 4: CompletableFuture

4.1 Definition & Concepts

CompletableFuture: A CompletableFuture is used for asynchronous programming in Java. It represents a future result that can be completed manually and supports chaining, composition, and combination of async operations.

Evolution:

- Future (Java 5): Blocking, no callbacks
- FutureTask (Java 5): Manual completion
- CompletableFuture (Java 8): Non-blocking, callbacks, composition

4.2 Internal Working

```java
public class CompletableFuture<T> implements Future<T>, CompletionStage<T> {
    volatile Object result; // Either value or AltResult (exception)
    volatile Completion stack; // Linked list of dependent actions

    // Internal structure:
    // CompletableFuture<String> future = new CompletableFuture<>();
    //
    // result = null (pending)
    // stack = [thenApply, thenAccept, thenCompose...]
    //
    // When complete(value) called:
    // 1. CAS result from null to value
    // 2. Pop stack and execute each completion
    // 3. Each completion triggers next in chain

    static class UniApply<T,V> extends UniCompletion<T,V> {
        Function<? super T,? extends V> fn;

        void tryFire(int mode) {
            if (fn != null) {
                V result = fn.apply(dep.result);
                postComplete(result);
            }
        }
    }
}
```

Key Internal Mechanisms:

1. **Completion Stack**: LIFO stack of dependent actions
2. **CAS Operations**: Thread-safe completion
3. **Asynchronous Execution**: Uses ForkJoinPool.commonPool() by default
4. **Exception Propagation**: Wraps exceptions in `CompletionException`

4.3 Core Methods

Creation Methods

```java
// Already completed
CompletableFuture.completedFuture(value)

// Async supply (no result)
CompletableFuture<Void> runAsync(Runnable)
CompletableFuture<Void> runAsync(Runnable, Executor)

// Async supply (with result)
CompletableFuture<T> supplyAsync(Supplier<T>)
CompletableFuture<T> supplyAsync(Supplier<T>, Executor)
```

Transformation Methods

```java
// ThenApply: Transform result (sync)
<U> CompletableFuture<U> thenApply(Function<T,U>)

// ThenApplyAsync: Transform result (async)
<U> CompletableFuture<U> thenApplyAsync(Function<T,U>)

// ThenCompose: Chain futures (flatmap)
<U> CompletableFuture<U> thenCompose(Function<T,CompletableFuture<U>>)

// ThenAccept: Consume result (no return)
CompletableFuture<Void> thenAccept(Consumer<T>)
```

Combination Methods

```java
// thenCombine: Combine two futures
<U,V> CompletableFuture<V> thenCombine(
    CompletionStage<U>,
    BiFunction<T,U,V>
)

// thenAcceptBoth: Consume two results
<U> CompletableFuture<Void> thenAcceptBoth(
    CompletionStage<U>,
    BiConsumer<T,U>
)

// applyToEither: Use first to complete
<U> CompletableFuture<U> applyToEither(
    CompletionStage<T>,
    Function<T,U>
)
```

Aggregation Methods

```java
// allOf: Wait for all
static CompletableFuture<Void> allOf(CompletableFuture<?>...)

// anyOf: Wait for first
static CompletableFuture<Object> anyOf(CompletableFuture<?>...)
```

Exception Handling

```java
// handle: Handle both success and exception
<U> CompletableFuture<U> handle(BiFunction<T,Throwable,U>)

// exceptionally: Handle only exception
CompletableFuture<T> exceptionally(Function<Throwable,T>)

// whenComplete: Side effect on completion
CompletableFuture<T> whenComplete(BiConsumer<T,Throwable>)
```

Manual Completion

```java
boolean complete(T value)
boolean completeExceptionally(Throwable ex)
boolean cancel(boolean mayInterrupt)
```

4.4 Execution Flow

```java
// Example chain:
CompletableFuture.supplyAsync(() -> fetchUser(id))        // Stage 1
    .thenApply(user -> validate(user))                     // Stage 2
    .thenCompose(validUser -> saveAsync(validUser))        // Stage 3
    .thenAccept(saved -> log(saved))                       // Stage 4
    .exceptionally(ex -> handleError(ex));                 // Error handler

// Internal flow:
// 1. supplyAsync completes -> triggers thenApply
// 2. thenApply completes -> triggers thenCompose
// 3. thenCompose completes -> triggers thenAccept
// 4. If any throws -> exceptionally catches
```

4.5 When to Use

- **Async I/O operations**: HTTP calls, database queries
- **Pipeline processing**: Transform data through stages
- **Parallel composition**: Run multiple independent tasks

- **Non-blocking code**: Avoid callback hell

4.6 Simple Example

```
public class CompletableFutureExample {
    public static void main(String[] args) throws Exception {
     // 1. Simple async computation
     CompletableFuture<Integer> future = CompletableFuture
        .supplyAsync(() -> {
            System.out.println("Computing...");
            return 42;
        });
            // 2. Chain transformations
     CompletableFuture<String> result = future
        .thenApply(n -> n * 2)              // 84
        .thenApply(n -> "Result: " + n)
        .thenApply(String::toUpperCase);

     System.out.println(result.get());
            // 3. Combine two futures
     CompletableFuture<Integer> future1 = CompletableFuture.supplyAsync(() -> 10);
     CompletableFuture<Integer> future2 = CompletableFuture.supplyAsync(() -> 20);

     CompletableFuture<Integer> combined = future1
        .thenCombine(future2, (a, b) -> a + b);
            System.out.println("Combined: " + combined.get());
            // 4. AllOf - wait for all
     CompletableFuture<Void> all = CompletableFuture.allOf(future1, future2);
     all.join(); // Wait for all
            // 5. Exception handling
     CompletableFuture<Integer> exceptional = CompletableFuture
        .supplyAsync(() -> {
```

```
            if (true) throw new RuntimeException("Error!");
            return 100;
        })
        .exceptionally(ex -> {
            System.out.println("Handled: " + ex.getMessage());
            return 0;
        });
            System.out.println("Exception result: " + exceptional.get());
            // 6. Real-world example: Fetch user, then orders, then details
    CompletableFuture
        .supplyAsync(() -> fetchUser(1))
        .thenCompose(user -> fetchOrders(user.id)
            .thenApply(orders -> new UserWithOrders(user, orders)))
        .thenAccept(result -> System.out.println(result))
        .exceptionally(ex -> {
            System.err.println("Failed: " + ex.getMessage());
            return null;
        });

    Thread.sleep(2000);
  }
    static class User { int id; String name; }
  static class Order { int id; }
  static class UserWithOrders { User user; List<Order> orders; }
    static User fetchUser(int id) { return new User(); }
  static CompletableFuture<List<Order>> fetchOrders(int userId) {
    return CompletableFuture.supplyAsync(() -> List.of(new Order()));
  }
}
```

Synchronizer	Reusable	Dynamic Parties	Phases	Best For
CountDownLatch	No	No	Single	One-time wait
CyclicBarrier	Yes	No	Multiple	Reusable barrier
Semaphore	Yes	N/A	N/A	Resource limiting
Phaser	Yes	Yes	Multiple	Dynamic coordination

Future Type	Blocking	Callbacks	Composition	Manual Complete
Future	Yes	No	No	No
CompletableFuture	No	Yes	Yes	Yes

Executor	Use Case
ForkJoinPool	CPU-intensive, divide-and-conquer
ThreadPoolExecutor	General purpose, I/O-bound
ScheduledThreadPool	Delayed/periodic tasks

1. clone() and Cloneable – Shallow Copy in Java

Cloneable Interface

The **Cloneable interface** in Java is a **marker interface** that allows objects to be cloned.

If a class does **not implement Cloneable**, calling clone() throws **CloneNotSupportedException**.

---

clone() Method

The clone() method is defined in **Object class**.

protected Object clone() throws CloneNotSupportedException

It creates **a copy of an object**.

Example

```
class Student implements Cloneable {
  int id;
  String name;

  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }

  public Object clone() throws CloneNotSupportedException {
    return super.clone();
  }

  public static void main(String[] args) throws Exception {
    Student s1 = new Student(1,"Ravi");
    Student s2 = (Student) s1.clone();

    System.out.println(s1.id + " " + s1.name);
    System.out.println(s2.id + " " + s2.name);
  }
}
```

Shallow Copy

A **shallow copy** copies the **object but not the nested objects**.

```
Original Object
    |
```

```
    v
  Object A ------> Address

Clone Object
    |
    v
  Object B ------> Address (same reference)
```

Both objects **share the same reference for inner objects**.

Shallow Copy Example

```
class Address {
  String city;
}

class Person implements Cloneable {
  Address address;

  protected Object clone() throws CloneNotSupportedException {
    return super.clone();
  }
}
```

Here, the **address object is shared**.

---

2. Annotations in Java

Annotations provide **metadata about code**.

They are introduced in **Java 5**.

Annotations **do not change program logic** but give instructions to the compiler or tools.

Built-in Annotations

1️⃣ **@Override**

Indicates that a method overrides a superclass method.

```
class Animal {
  void sound() {}
}

class Dog extends Animal {
```

```
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

2️⃣ @Deprecated

Marks a method or class as **obsolete**.

```
@Deprecated
void oldMethod() {
}
```

3️⃣ @SuppressWarnings

Suppresses compiler warnings.

```
@SuppressWarnings("unchecked")
```

Annotation Diagram

```
Annotations
   |
 -----------------------
|       |       |
@Override  @Deprecated  @SuppressWarnings
```

---

3. Assertions in Java

Assertions are used for **debugging and testing assumptions**.

If the condition is **false**, the program throws **AssertionError**.

Syntax

```
assert condition;
```

or

```
assert condition : "Error message";
```

Example

```
int age = 15;
assert age >= 18 : "Not eligible to vote";
```

If age < 18 → **AssertionError occurs**.

Enabling Assertions

Assertions are **disabled by default**.

Enable them using JVM option:

```
java -ea Test
```

Assertion Diagram

```
Program
  |
  v
assert condition
  |
true -------> Continue
false -----> AssertionError
```

---

4. Class Loader in Java

The **ClassLoader** loads Java classes into the JVM.

When a Java program runs, **classes are loaded dynamically**.

Types of Class Loaders

1️⃣ **Bootstrap Class Loader**

Loads core Java classes from:

JAVA_HOME/lib

Example classes:

- **String**
- **Object**

2️⃣ **Extension Class Loader**

Loads classes from:

JAVA_HOME/lib/ext

3️⃣ Application Class Loader

Loads classes from:

CLASSPATH

These are **user-defined classes**.

Class Loader Hierarchy

```
Bootstrap ClassLoader
        |
        v
Extension ClassLoader
        |
        v
Application ClassLoader
```

Loading follows **Parent Delegation Model**.

Quick Summary Table

Topic	Purpose
clone()	Creates object copy
Cloneable	Marker interface for cloning
Shallow Copy	Copies object but shares references
Annotations	Metadata for code
Assertions	Debugging checks
Class Loader	Loads classes into JVM

Reflection API in Java

Definition

The **Reflection API** allows a Java program to **inspect and manipulate classes, methods, fields, and constructors at runtime**.

Using reflection, we can:

- Get information about a class
- Access private methods/fields
- Create objects dynamically
- Invoke methods at runtime

Reflection works mainly with classes from the **java.lang.reflect** package.

Main Classes in Reflection API

Important classes used in reflection:

Class	Purpose
Class	Represents a class
Method	Represents a method
Field	Represents a variable
Constructor	Represents a constructor

How to Get Class Object

There are **3 ways to get a Class object**.

1️⃣ **Using Class.forName()**

Class c = Class.forName("Student");

Loads the class dynamically.

2️⃣ **Using .class**

Class c = Student.class;

3️⃣ **Using getClass()**

```
Student s = new Student();
Class c = s.getClass();
```

Example of Reflection

```
import java.lang.reflect.Method;

class Student {

  public void display() {
    System.out.println("Hello Reflection");
  }
}

public class Test {

  public static void main(String[] args) throws Exception {

    Class c = Class.forName("Student");

    Object obj = c.newInstance();

    Method m = c.getMethod("display");

    m.invoke(obj);
  }
}
```

Output

Hello Reflection

Explanation:

1. Load class
2. Create object
3. Find method
4. Invoke method dynamically

Accessing Fields Using Reflection

```
import java.lang.reflect.Field;

class Person {
```

```
    private String name = "John";
}

public class Test {
    public static void main(String[] args) throws Exception {

        Person p = new Person();

        Field f = Person.class.getDeclaredField("name");

        f.setAccessible(true);

        System.out.println(f.get(p));
    }
}
```

This accesses **private field using reflection**.

Reflection API Structure

```
Reflection API
    |
 -------------------------
|       |        |
Class     Method     Field
          |
       Constructor
```

Advantages

✓ Inspect classes at runtime
✓ Dynamic object creation
✓ Access private members
✓ Used by frameworks

Frameworks like **Spring Framework**, **Hibernate**, and **JUnit** heavily use reflection.

Disadvantages

❌ Slower than normal method calls
❌ Breaks encapsulation
❌ Security restrictions

Real-world Uses

- Dependency Injection
- ORM frameworks
- Testing frameworks
- Annotation processing

Quick Summary

Concept	Meaning
Reflection	Inspect program at runtime
Class	Represents a class
Method	Represents method
Field	Represents variable
Constructor	Represents constructor

✅ **One-line definition for exams**

Reflection API is a feature in Java that allows **inspection and manipulation of classes, methods, fields, and constructors at runtime**.

SOLID Principles in Java

SOLID is a set of **five object-oriented design principles** that help developers create **maintainable, scalable, and flexible software**.

SOLID stands for:

1. **S — Single Responsibility Principle (SRP)**
2. **O — Open/Closed Principle (OCP)**
3. **L — Liskov Substitution Principle (LSP)**
4. **I — Interface Segregation Principle (ISP)**
5. **D — Dependency Inversion Principle (DIP)**

These principles are widely used in **object-oriented programming and Java frameworks** like Spring Framework.

---

1. Single Responsibility Principle (SRP)

Definition

A **class should have only one reason to change**, meaning it should have **only one responsibility**.

Bad Example ❌

```
class Report {
  public void generateReport() {
    // generate report
  }

  public void printReport() {
    // print report
  }

  public void saveToDatabase() {
    // save report
  }
}
```

Problem:

- One class handles **generation, printing, and storage**.

Good Example ✅

```
class ReportGenerator {
  public void generateReport() {
    // generate report
```

```
  }
}

class ReportPrinter {
  public void printReport() {
    // print report
  }
}

class ReportRepository {
  public void saveReport() {
    // save report
  }
}
```

Now each class has **one responsibility**.

Benefits

✓ Easier maintenance
✓ Better readability
✓ Easier testing

---

2. Open/Closed Principle (OCP)

Definition

Software entities should be:

- **Open for extension**
- **Closed for modification**

This means we should **add new behavior without changing existing code**.

Bad Example ❌

```
class Discount {
  public double giveDiscount(String customerType) {
    if(customerType.equals("Regular")) {
      return 10;
    } else if(customerType.equals("Premium")) {
      return 20;
    }
    return 0;
  }
}
```

Every new customer type requires **modifying the class**.

Good Example ✅

```
interface Discount {
  double giveDiscount();
}

class RegularCustomer implements Discount {
  public double giveDiscount() {
    return 10;
  }
}

class PremiumCustomer implements Discount {
  public double giveDiscount() {
    return 20;
  }
}
```

To add a new discount type, just **create another class**.

Benefits

✓ Easy to extend features
✓ Reduces risk of breaking existing code

---

3. Liskov Substitution Principle (LSP)

Definition

Objects of a **subclass should be replaceable with objects of the superclass without breaking the program**.

Bad Example ❌

```
class Bird {
  void fly() {}
}

class Penguin extends Bird {
  void fly() {
    throw new UnsupportedOperationException();
  }
}
```

Problem:

- Penguins **cannot fly**, but the class forces it.

Good Example ✅

```
class Bird {}

class FlyingBird extends Bird {
  void fly() {}
}

class Sparrow extends FlyingBird {}

class Penguin extends Bird {}
```

Now the design correctly models reality.

Benefits

✓ Proper inheritance
✓ Avoids unexpected behavior

---

4. Interface Segregation Principle (ISP)

Definition

A class should **not be forced to implement interfaces it does not use**.

Bad Example ❌

```
interface Worker {
  void work();
  void eat();
}

class Robot implements Worker {
  public void work() {}

  public void eat() {
    // not applicable
  }
}
```

Problem:
Robots **don't eat**, but must implement the method.

Good Example ✅

```
interface Workable {
  void work();
}
```

```
interface Eatable {
  void eat();
}

class Human implements Workable, Eatable {
  public void work() {}
  public void eat() {}
}

class Robot implements Workable {
  public void work() {}
}
```

Benefits

✓ Clean interfaces
✓ Avoid unnecessary methods

---

5. Dependency Inversion Principle (DIP)

Definition

1. High-level modules should **not depend on low-level modules**
2. Both should depend on **abstractions**

Bad Example ❌

```
class Keyboard {}

class Computer {
  Keyboard keyboard = new Keyboard();
}
```

The **Computer class is tightly coupled** to Keyboard.

Good Example ✅

```
interface InputDevice {}

class Keyboard implements InputDevice {}

class Computer {
  InputDevice device;

  Computer(InputDevice device) {
    this.device = device;
```

```
  }
}
```

Now we can plug **Keyboard, Mouse, TouchScreen**, etc.

Benefits

✓ Loose coupling
✓ Easier testing
✓ Better scalability

This principle is heavily used in frameworks like Spring Boot through **dependency injection**.

Summary Table

Principle	Meaning
SRP	One class → One responsibility
OCP	Extend behavior without modifying code
LSP	Subclasses must behave like parent classes
ISP	Create small, specific interfaces
DIP	Depend on abstractions, not concrete classes

✅ **In simple words**

- **SRP** → One job per class
- **OCP** → Add new features without changing old code
- **LSP** → Child classes should properly replace parent classes
- **ISP** → Don't force classes to implement unused methods
- **DIP** → Use interfaces instead of concrete classes

Design Patterns in Java

What is a Design Pattern?

A **design pattern** is a **reusable solution to a commonly occurring problem in software design**.

Design patterns are **not code**, but **templates or best practices** used when designing systems.

Benefits

✓ Reusable solutions
✓ Improves code readability
✓ Makes code flexible and maintainable
✓ Follows object-oriented principles (like SOLID)

Types of Design Patterns

Design patterns are divided into **three main categories**.

Category	Purpose
Creational	Object creation mechanisms
Structural	Class and object relationships
Behavioral	Communication between objects

---

1. Creational Design Patterns

These patterns deal with **object creation**.

Types

1. **Singleton Pattern**
2. **Factory Method Pattern**
3. **Abstract Factory Pattern**
4. **Builder Pattern**
5. **Prototype Pattern**

Singleton Pattern

Definition

Ensures **only one instance of a class exists** and provides a **global access point**.

Example

```
class Singleton {

  private static Singleton instance;

  private Singleton() {}

  public static Singleton getInstance() {
    if(instance == null) {
      instance = new Singleton();
    }
    return instance;
  }
}
```

Use Cases

- Database connection
- Logging system
- Configuration manager

Singleton Pattern

```
+------------------+
|    Singleton     |
+------------------+
| - instance       |
+------------------+
| + getInstance()  |
+------------------+

        |
        v
  Only ONE Object
```

Idea: Only **one instance** of a class exists.

Factory Pattern

Definition

Creates objects **without exposing the creation logic**.

Example

```
interface Shape {
  void draw();
}

class Circle implements Shape {
  public void draw() {
    System.out.println("Circle");
```

```
  }
}

class Square implements Shape {
  public void draw() {
    System.out.println("Square");
  }
}

class ShapeFactory {

  public Shape getShape(String type) {
    if(type.equals("circle"))
      return new Circle();
    else
      return new Square();
  }
}
```

Benefit

Client code **does not need to know the exact class**.

Factory Pattern

```
        Client
           |
           v
     +------------+
     |  Factory   |
     +------------+
       /        \
      v          v
  +-------+   +-------+
  |Circle |   |Square |
  +-------+   +-------+
```

Idea: Factory decides **which object to create.**

Builder Pattern

Definition

Used to **construct complex objects step by step**.

Example

```
class Computer {

  private String CPU;
  private String RAM;

  public Computer(String CPU, String RAM) {
```

```
    this.CPU = CPU;
    this.RAM = RAM;
  }
}

class ComputerBuilder {

  private String CPU;
  private String RAM;

  public ComputerBuilder setCPU(String CPU) {
    this.CPU = CPU;
    return this;
  }

  public ComputerBuilder setRAM(String RAM) {
    this.RAM = RAM;
    return this;
  }

  public Computer build() {
    return new Computer(CPU, RAM);
  }
}
```

Usage:

```
Computer pc = new ComputerBuilder()
        .setCPU("Intel")
        .setRAM("16GB")
        .build();
```

Builder Pattern

```
    Client
       |
       v
  +---------+
  | Builder |
  +---------+
    |   |   |
    v   v   v
  CPU RAM HDD
       |
       v
   +---------+
   | Product |
   +---------+
```

Idea: Build **complex objects step-by-step**.

2. Structural Design Patterns

These patterns deal with **how classes and objects are structured**.

Types

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Facade
6. Flyweight
7. Proxy

Adapter Pattern

Definition

Allows **incompatible interfaces to work together**.

Example:
When connecting **new system with legacy code**.

Example

```
interface MediaPlayer {
  void play(String audioType);
}
class Mp3Player implements MediaPlayer {
  public void play(String audioType) {
    System.out.println("Playing mp3");
  }
}
```

Adapter converts one interface into another.

Adapter Pattern

```
Client
  |
  v
+--------+
|Adapter |
+--------+
     |
     v
+-----------+
| OldSystem |
+-----------+
```

Idea: Makes **incompatible interfaces work together**.

Decorator Pattern

Definition

Adds **new functionality to an object dynamically** without modifying its structure.

Example:
Adding features like **extra toppings in pizza**.

Decorator Pattern

```
        +-----------+
        | Component |
        +-----------+
              ^
              |
       +-------------+
       | Decorator   |
       +-------------+
        /          \
+------------+  +------------+
| DecoratorA |  | DecoratorB |
+------------+  +------------+
```

Idea: Add extra features dynamically.

Facade Pattern

Definition

Provides a **simplified interface to a complex system**.

Example:
A **single API that interacts with many subsystems**.

Frameworks like Hibernate often expose simplified APIs using facade-like ideas.

Facade Pattern

```
        Client
           |
           v
        +------+
        |Facade|
        +------+
        /   |   \
       v    v    v
  System1 System2 System3
```

Idea: Simplifies complex systems.

---

3. Behavioral Design Patterns

These patterns focus on **communication between objects**.

Types

1. Observer
2. Strategy

3. Command
4. State
5. Template Method
6. Iterator
7. Mediator
8. Chain of Responsibility
9. Visitor
10. Interpreter
11. Memento

Observer Pattern

Definition

Defines a **one-to-many dependency** between objects so when **one object changes state**, all dependents are notified.

Example

```
interface Observer {
  void update();
}

class Subscriber implements Observer {
  public void update() {
    System.out.println("Notification received");
  }
}
```

Real-world example:

- YouTube subscriptions
- Notification systems

Observer Pattern

```
        +---------+
        | Subject |
        +---------+
         /   |   \
        v    v    v
+---------+ +---------+ +---------+
|Observer1| |Observer2| |Observer3|
+---------+ +---------+ +---------+
```

Idea: When **subject changes**, observers get notified.

Example: **Notification system**

-

Strategy Pattern

Definition

Defines a **family of algorithms**, encapsulates them, and makes them interchangeable.

Example

```
interface PaymentStrategy {
  void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
  public void pay(int amount) {
    System.out.println("Paid using credit card");
  }
}

class UpiPayment implements PaymentStrategy {
  public void pay(int amount) {
    System.out.println("Paid using UPI");
  }
}
```

Usage:

```
PaymentStrategy payment = new UpiPayment();
payment.pay(500);
```

Strategy Pattern

```
        Client
          |
          v
      +---------+
      |Strategy |
      +---------+
        /     \
       v       v
+------------+ +------------+
| Strategy A | | Strategy B |
+------------+ +------------+
```

Idea: Choose **algorithm at runtime**.

Example: Payment method.

Complete List of GoF Design Patterns

Creational (5)

- Singleton
- Factory Method

- Abstract Factory
- Builder
- Prototype

Structural (7)

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Behavioral (11)

- Observer
- Strategy
- Command
- State
- Template Method
- Iterator
- Mediator
- Chain of Responsibility
- Visitor
- Interpreter
- Memento

Total = **23 Design Patterns**

Real-World Usage in Java

Many Java libraries use design patterns.

Examples:

- Spring Framework → Dependency Injection (Factory pattern)
- Hibernate → DAO pattern

- Java Collections Framework → Iterator pattern

✅ Simple Memory Trick

Category Remember

Creational Object creation

Structural Object structure

Behavioral Object communication

Command Pattern

```
Client
  |
  v
+--------+
|Command |
+--------+
    |
    v
+---------+
|Receiver |
+---------+
```

Idea: Encapsulates **requests as objects.**

Super Short Summary Diagram

```
Design Patterns
     |
  ------------------------
  |          |           |
Creational Structural Behavioral
  |          |           |
Singleton   Adapter     Observer
Factory     Decorator   Strategy
Builder     Facade      Command
```